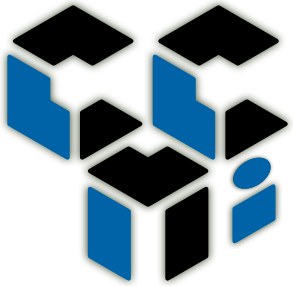
OntoUML: Shrnutí, anti-patterny





FAKULTA
INFORMAČNÍCH
TECHNOLOGIÍ
ČVUT V PRAZE

BI-KOM: Konceptuální modelování #5

doc. Ing. Robert Pergl, Ph.D. (robert.pergl@fit.cvut.cz),
Ing. Marek Bělohoubek (belohma2@fit.cvut.cz)

Pozn.: Pro přípravu přednášky byly mj. využity dostupné materiály prof. Giancarla Guizzardiho.

Obsah přednášky

1. Několik poznámek k OntoUML (UFO-A)
2. Kontrola modelu - anti-patterny
3. Naše ukázkové projekty OntoUML/UFO-A

Terminologický koutek

- Ontologické modelování (OntoUML):
  - *Entita (Entity)*: abstrakce věci reálného (hmotného i nehmotného) světa = "krabička v diagramu" (např. Člověk)
  - *Instance (Instance)*: konkrétní věc reálného světa, instance entity (např. Marek)
  - *Typ entity (Entity Type)*: typ definující ontologické vlastnosti entity = stereotyp "krabičky v diagramu" (např. «Kind»)
  - *Objekt (Object)*: existenčně nezávislý typ entity (vs Aspekt)
- Softwarově-inženýrský model objektového systému (UML):
  - *Třída (Class)*: vzor pro instanciaci objektů, obsahuje schéma atributů a implementaci metod
  - *Objekt, Instance (Object, Instance)*: instance třídy vytvořená konstruktorem (typicky `new()`)

Terminologický koutek

- Aby zmatení bylo ještě dokonalejší, v OntoUML jakožto profilu UML mluvíme o *Diagramu tříd (Class Diagram)*, byť by to tedy správně měl být *Diagram typů entit (Entity Type Diagram)*.



The Tower of Babel

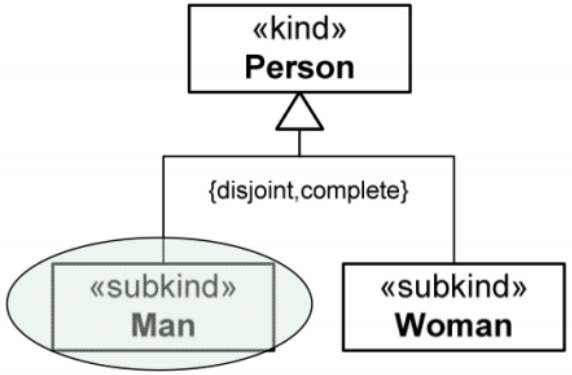
Postup modelování: Páteř identity

- Nejprve určíme, co jsou jedinci a co zachovává jejich identitu.
- Zvolíme vhodného poskytovatele identity: `Kind`, `Collective`, `Quantity`, `Relator`, `Mode` nebo `Quality`.
- Přidáme `Subkind` pro rigidní specializace, poté `Phase` a `Role` pro anti-rigidní.
- Každý sortál dědící identitu musí vést k právě jednomu poskytovateli, a to i při vícenásobném dědění.

OntoUML.org: identita · Subkind

Postup modelování: Rigidní specializace

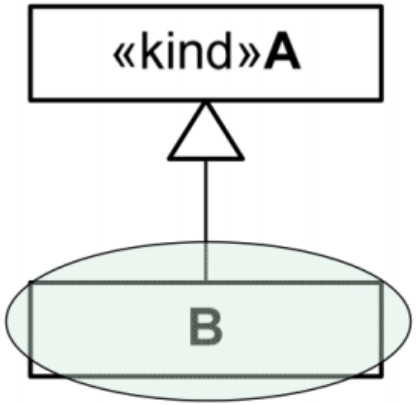
- `Subkind` dědí identitu a přidává esenciální rozlišující vlastnost.
- Může přímo či nepřímo specializovat kteréhokoli poskytovatele identity.
- Hledáme související podtypy, ale `disjoint` a `complete` volíme podle domény; partice není povinná.



Existuje nějaký další SubKind Kindu Person, který je komplementární k Man?

Postup modelování: Identifikace Anti-Rigid Sortal

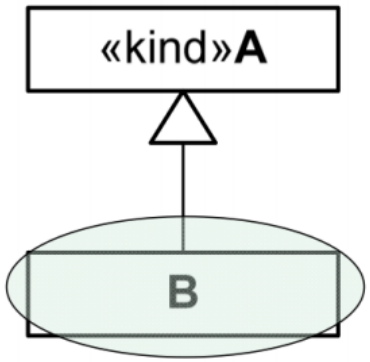
- Může se **[A]**, které není **[B]**, stát **[B]**?
  (např. může se Člověk, který není Otec, stát Otcem?)
- Může **[A]**, které je **[B]**, přestat býti **[B]**?
  (např. může Člověk, který je Otec, přestat být Otec?).



Kandidátní
Anti-Rigidní třída

Postup modelování: Identifikace «Role»

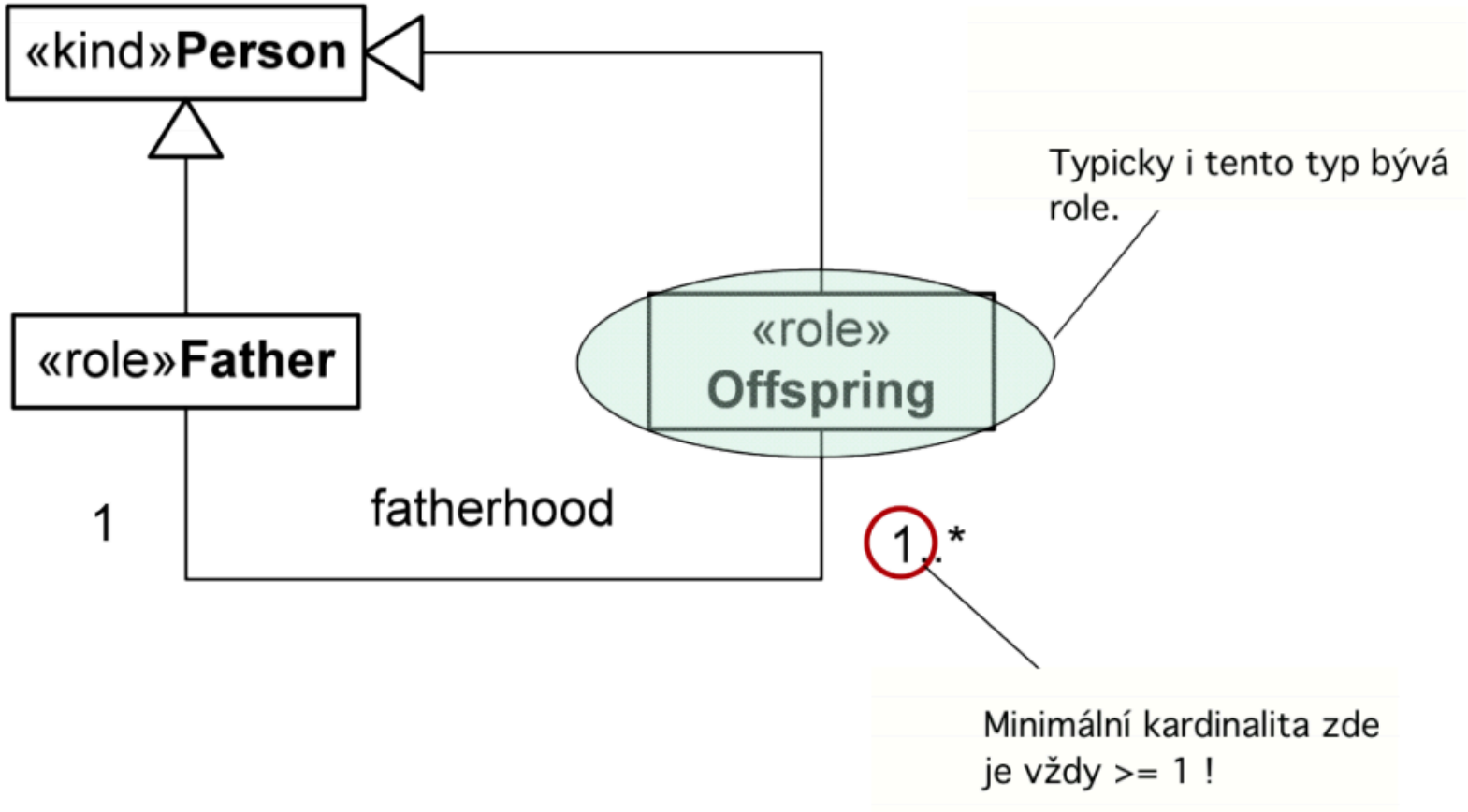
- `Role` dědí právě jeden princip identity a klasifikuje jedince na základě relační podmínky. Její poskytovatel nemusí být `Kind`.



Anti-Rigidní třída

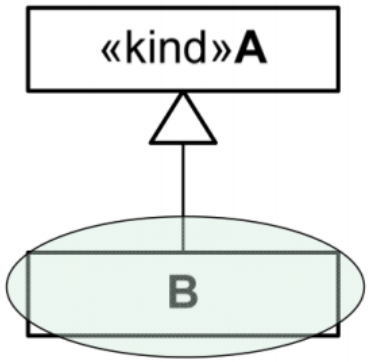
- Stane se **[A] [B]** tím, že naváže relaci s jiným typem entity? (např. stane se Člověk Otcem tím, že naváže relace k jinému typu entity?).
- Jak se bude jmenovat tato definující relace?
- Najděme kardinality a parciality této definující relace.

Postup modelování: Identifikace «Role»



Postup modelování: Identifikace «Phase»

- `Phase` dědí právě jeden princip identity a klasifikuje jedince na základě intrinsické podmínky. Její poskytovatel nemusí být `Kind`.



Anti-Rigidní třída

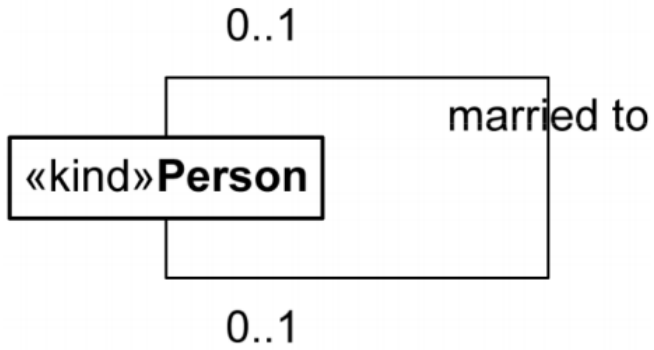
- Stane se **[A] [B]** kvůli změně jedné nebo vícero hodnot atributů? (např. stane se Člověk Teenagerem kvůli změně jedné nebo vícero hodnot atributů?).
- Najděme definující atributy a jejich definující hodnoty.

Postup modelování: Identifikace non-sortálů

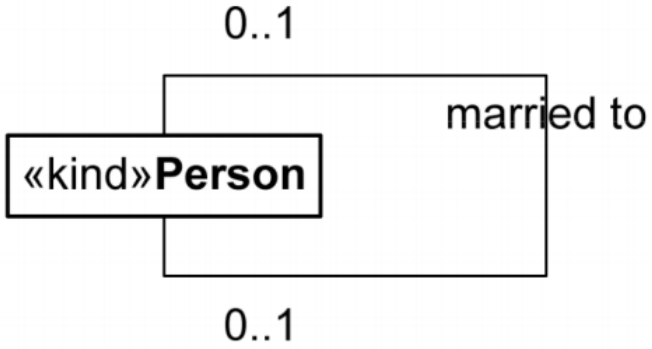
- Non-sortály reprezentují společné rysy, které jsou sdíleny entitami vícero typů.
- Non-sortály tedy hrají roli **strukturujícího mechanismu** v konceptuálních modelech (obzvláště v rozsáhlých ontologiích).
- Vytváříme tedy nový, abstraktní nadtyp nad existujícími sortály.
- Musíme "pouze" vybrat správný typ non-sortálu, který bude sedět z hlediska definic a vlastností (viz "Neformální shrnutí OntoUML: Non-sortály").

Postup modelování: Princip reifikace

Reifikace (reification) je postup, kdy do modelu zavádíme dosud nevyjádřené pojmy, model "zpřesňujeme" a zvyšujeme výrazovost (expressivity).

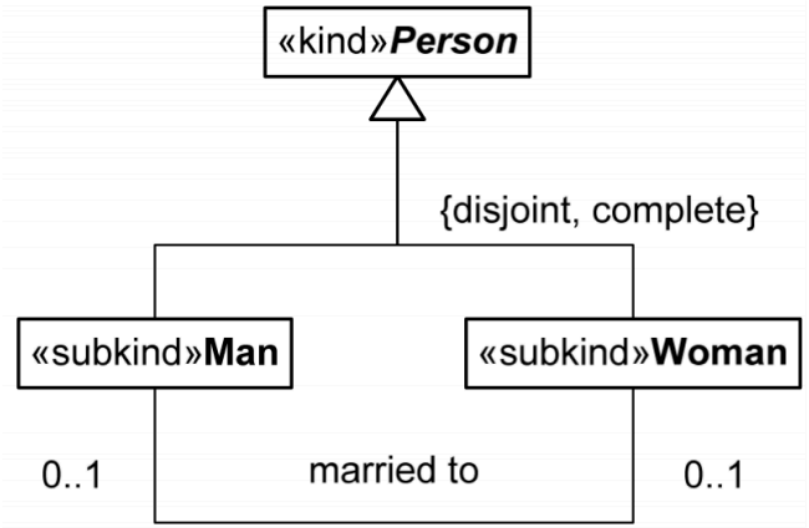


Postup modelování: Princip reifikace



OK, svazek může nastat mezi dvěma Lidmi, ale nyní budeme vyžadovat svazek pouze mezi Mužem a Ženou...

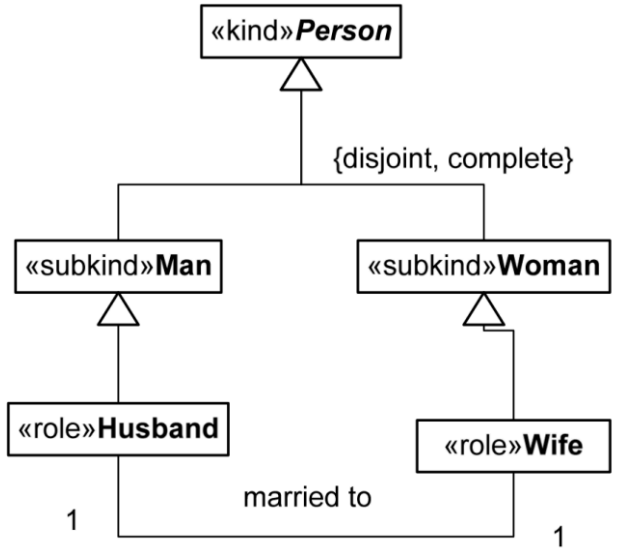
Postup modelování: Princip reifikace



OK, ale je to nepřesné: vdává se přeci Ženich a Nevěsta, že?

Tyto pojmy jsou "skryté" v Man/Woman, což má za následek multiplicitu `0..`.

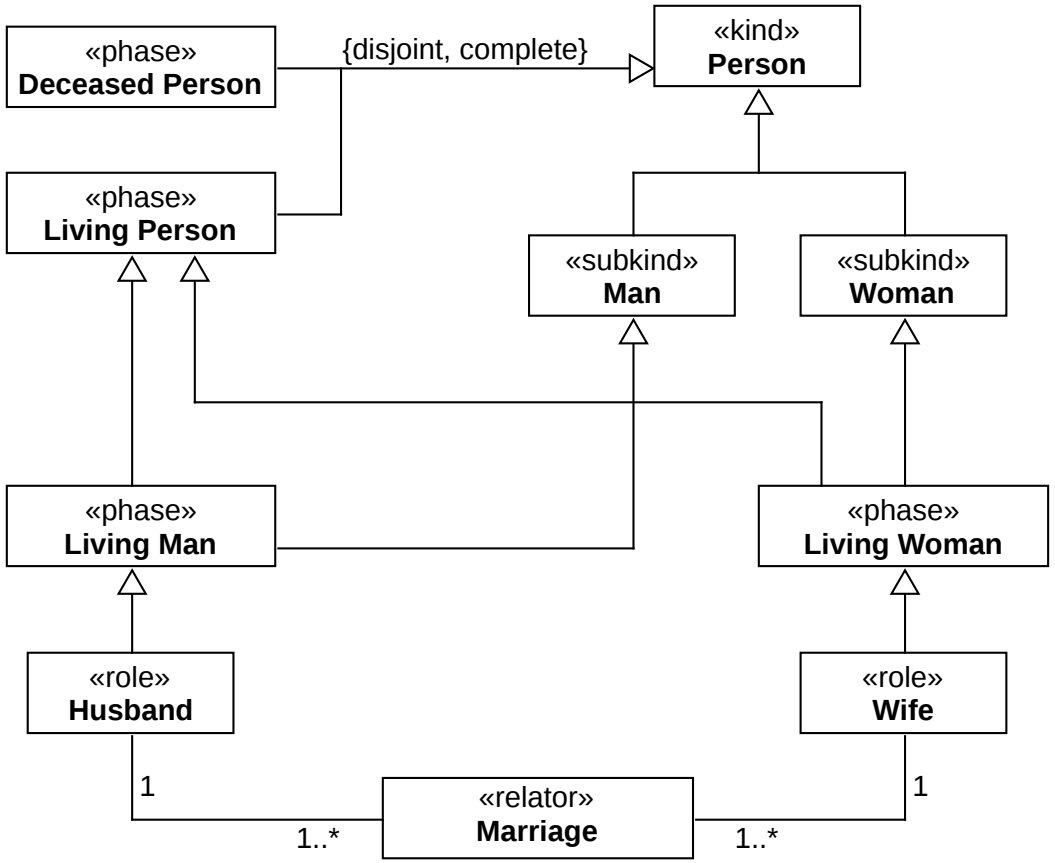
Postup modelování: Princip reifikace



Zde vyjádření rolí mění `0..1` na `1`. Volitelnost vybízí k otázce: nechybí role či vztah s vlastní identitou, dobou trvání nebo vlastnostmi? **Samotná volitelnost nerozhoduje.**

Fajn, ale dalo by se rýpnout, že takto se mohou vzít i mrtví lidé (nechceme Zombie Land)

Postup modelování: Princip reifikace



Kontrola modelu

Jak kontrolujeme modely?

- **Verifikace:** kontrola pravidel jazyka, např. dovolených generalizací a povinných závislostí. Porušení je nutné opravit.
- **Detekce anti-patternů:** odhaluje podezřelé struktury a vybízí k otázce, jaké nežádoucí situace umožňují. Nález může být záměrný.
- **Validace s doménou:** ověřujeme příklady i protipříklady s doménovým expertem. Nástroj nerozhodne, zda model vystihuje zamýšlenou doménu.

Syntaktická kontrola

- "Může být entita se stereotypem X podtypem entity se stereotypem Y?"
- "Může mít asociace se stereotypem A jako zdroj entitu se stereotypem X?"
- "Je každá Phase součástí disjoint a complete generalization setu?"
- "Mají všechny entity stereotypované jako role splněnou relační závislost?"

Syntaktická kontrola - OpenPonk

- Kontrola na základě OntoUML specifikace.
- V OpenPonku reprezentována verifikacemi.
- Všechny syntaktické chyby je nutno opravit!

Sémantická kontrola

- "Může být entita X v relaci Y sama se sebou?"
- "Má nadtyp phase alespoň jeden atribut, quality nebo mode?"
- "Mají všechny domain formal asociace skutečně správný stereotyp?"
- "Nejsou součástí jednoho generalization setu, jehož nadtypem není mixin, jak rigidní, tak anti-rigidní podtypy?"

Sémantická kontrola - Anti-patterny

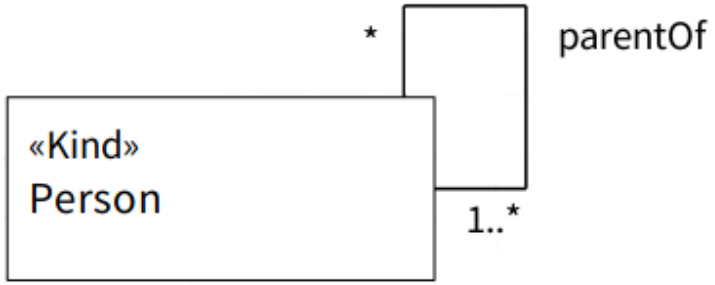
- Sémantická kontrola modelu.
- Detekce podezřelých struktur v modelu.
- V OpenPonku implementováno celkem dvacet anti-patternů.
- Nález vybízí k prověření; sám o sobě nedokazuje chybu.
- Plná dokumentace anti-patternů na ontouml.readthedocs.io/en/latest/anti-patterns/index.html

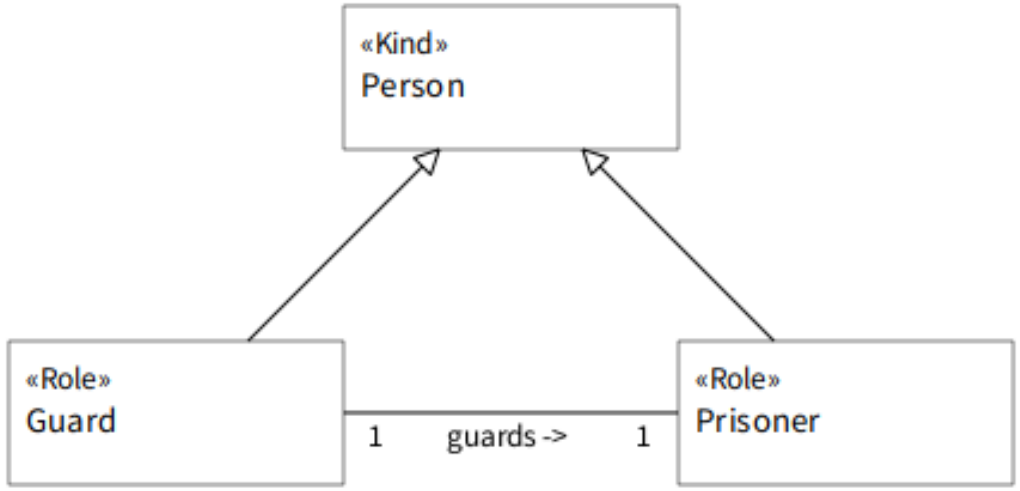
Anti-patterny - seznam

- Binary relation between overlapping types (BinOver)
- Relationally dependent phase (DepPhase)
- Free role specialization (FreeRole)
- Heterogeneous collective (HetColl)
- Mixin with the same rigidity (MixRig)
- Multiple relational dependency (MultDep)
- Relator mediating rigid types (RelRig)
- Undefined phase partition (UndefPhase)

Binary relation between overlapping types (BinOver)

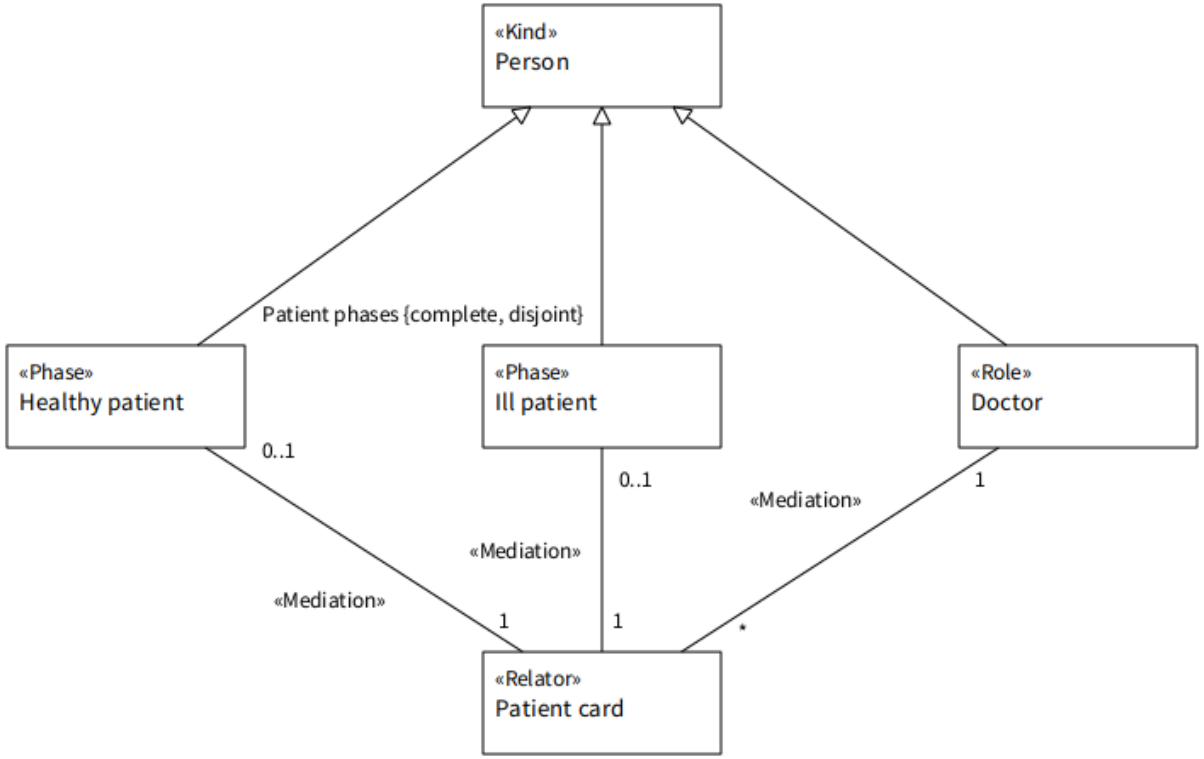
- Overlapping typy = dva typy, které mohou být instancovány stejným individualem.
- **"Na obou stranách relace může být stejný individual".**
- Existují další "...Over" anti-patterny (PartOver, RelOver, WholeOver).



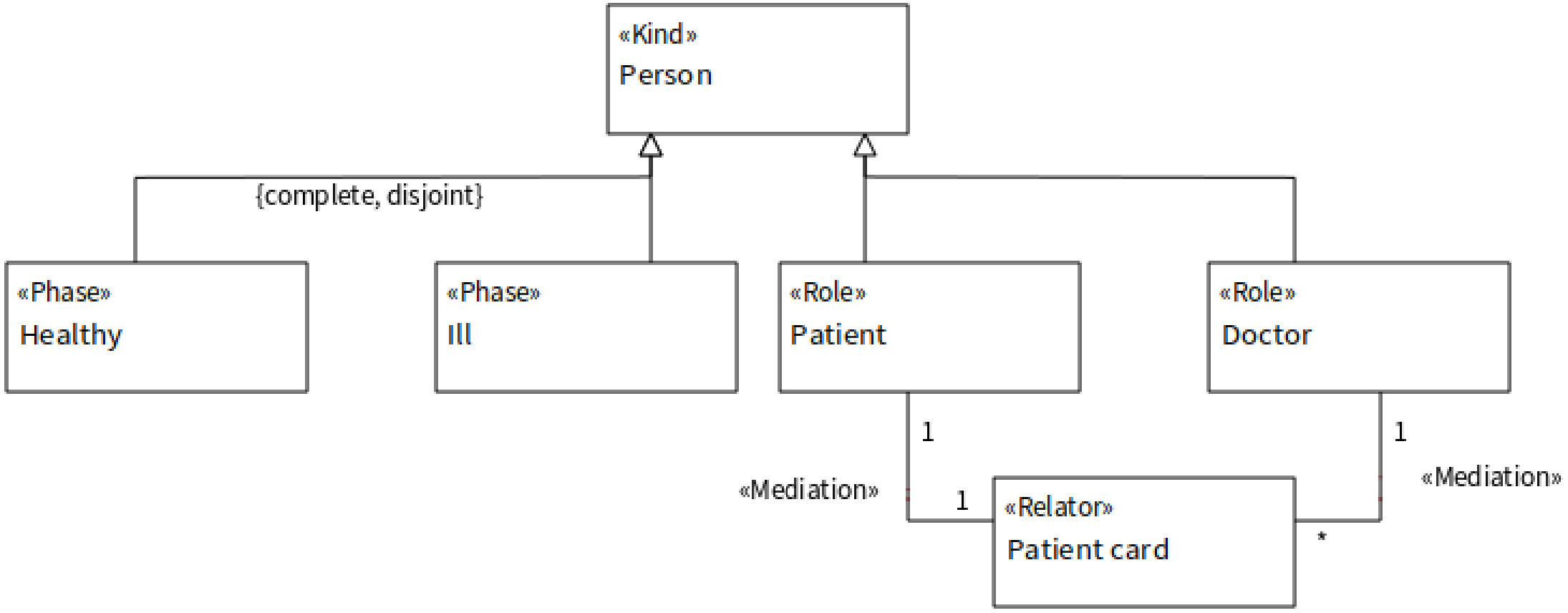


Relationally dependent phase (DepPhase)

- **Phase, která je součástí jedné nebo více asociací typu mediation.**
- Phase = závislá na změně vnitřního stavu.
- Role = závislá na existenci vnější asociace.

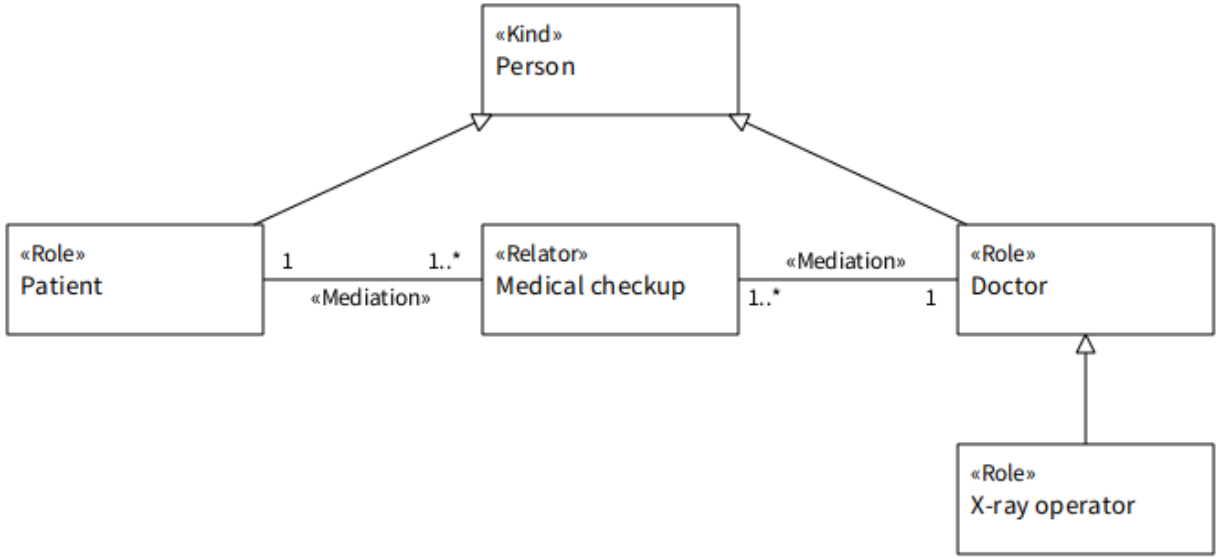


Relationally dependent phase (DepPhase) - oprava

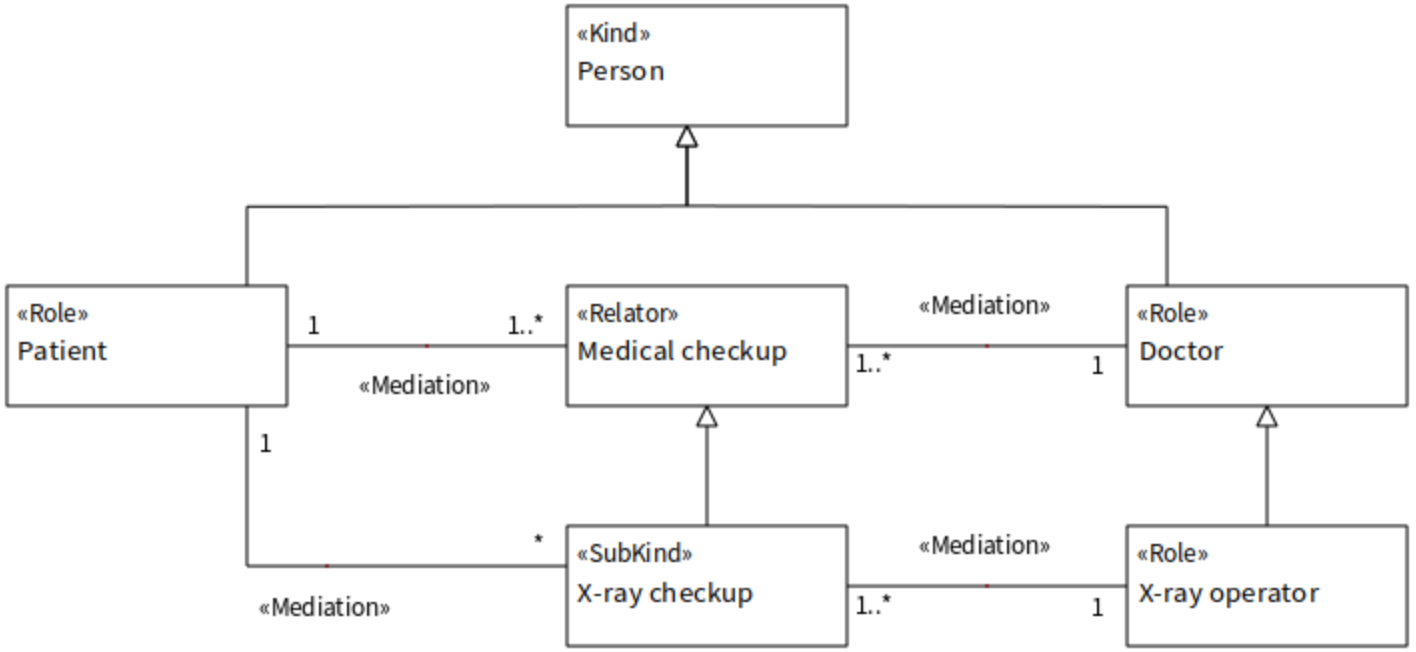


Free role specialization (FreeRole)

- Splnění relační závislosti role:
  - Role je součástí asociace stereotypované jako mediation.
  - Role je přímým podtypem rolemixinu, který je součástí mediation asociace.
  - **Role je podtypem jiné role, která má splněnou relační závislost**

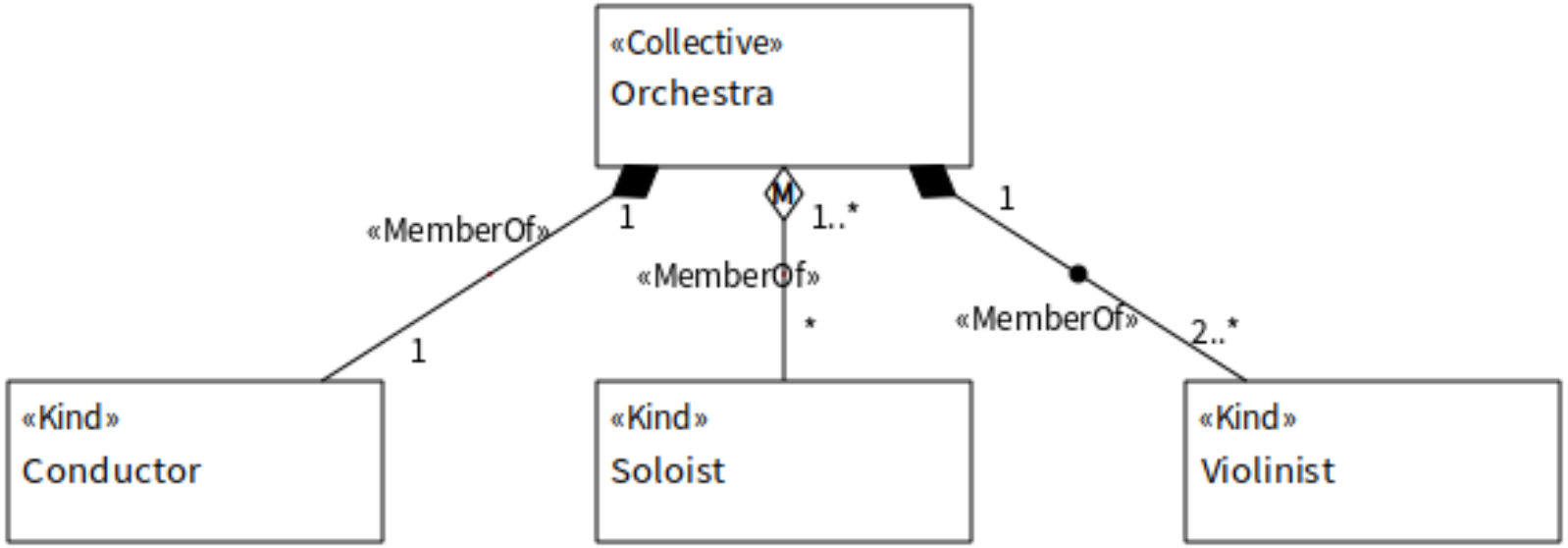


Free role specialization (FreeRole) - oprava

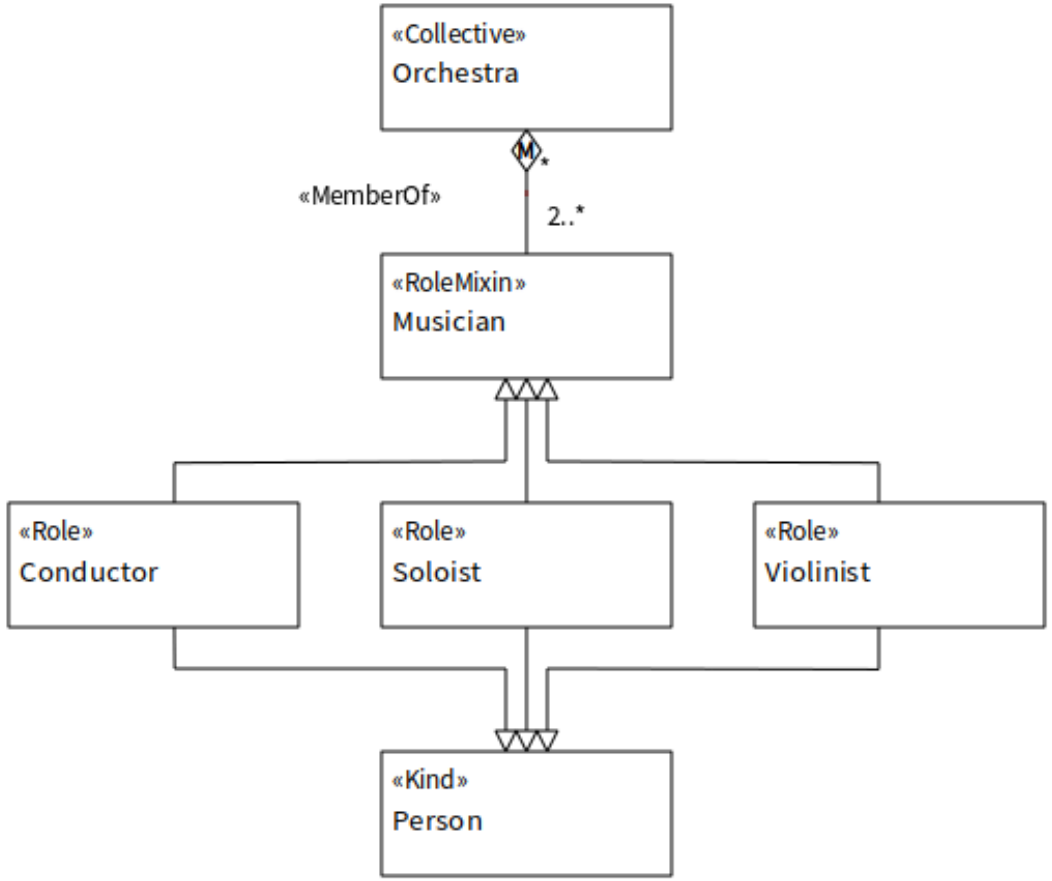


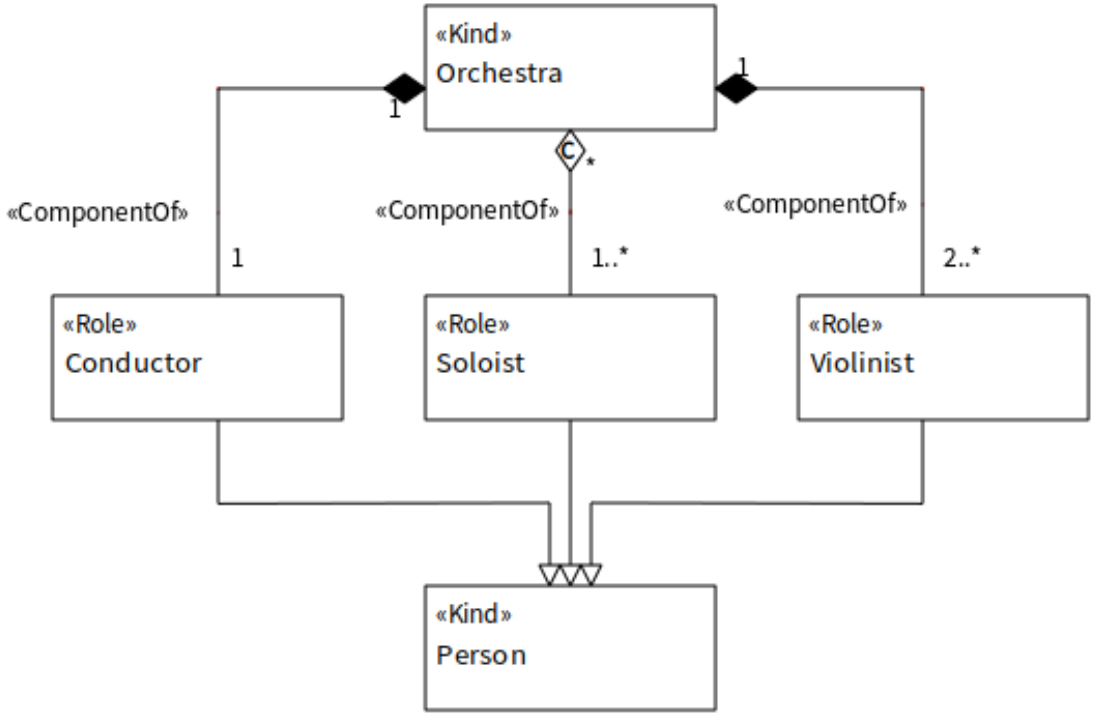
Heterogeneous collective (HetColl)

- Opakem je Homogeneous Functional Complex (HomoFunc).
- Collective = na všechny části je nahlíženo stejně.
- Funkční celek = jednotlivé části mají různé role.
- **Collective s dvěma či více rozdílnými typy částí.**



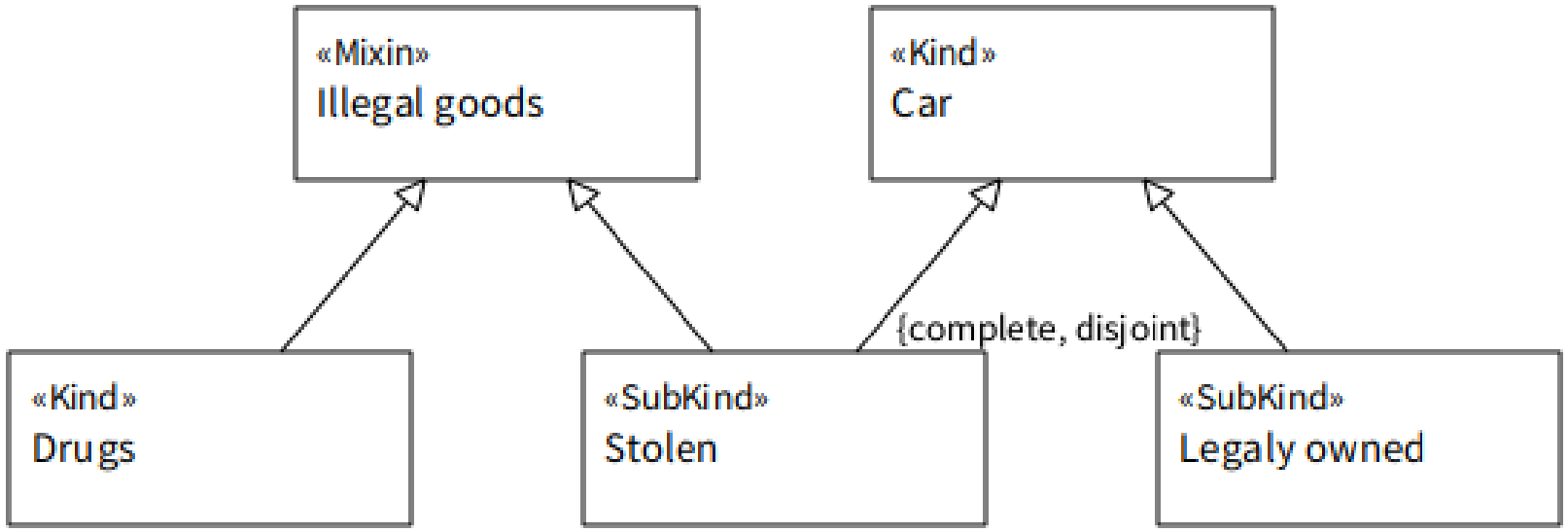
Heterogeneous collective (HetColl) - oprava



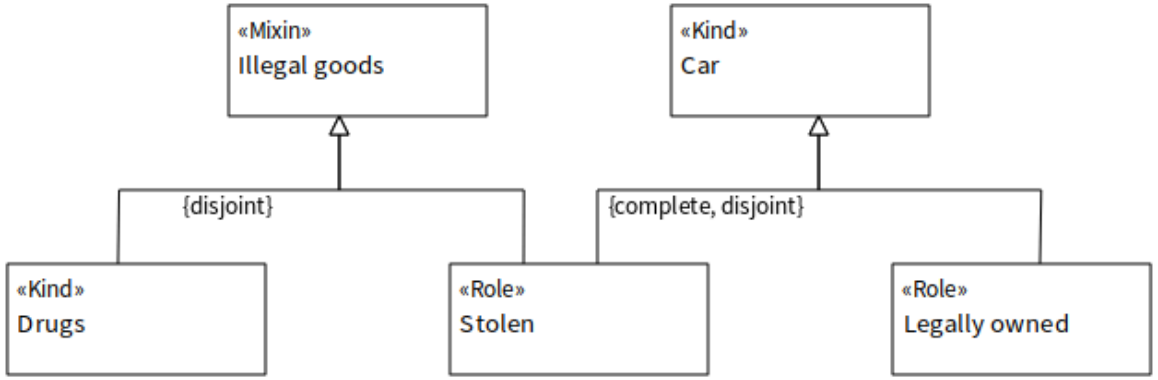


Mixin with the same rigidity (MixRig)

- Mixin může být nadtypem rigidních, anti-rigidních i semi-rigidních typů.
- Mixin musí mít minimálně dva podtypy s různou rigiditou.
- **Mixin, jehož podtypy jsou pouze rigidní, nebo pouze anti-rigidní podtypy.**

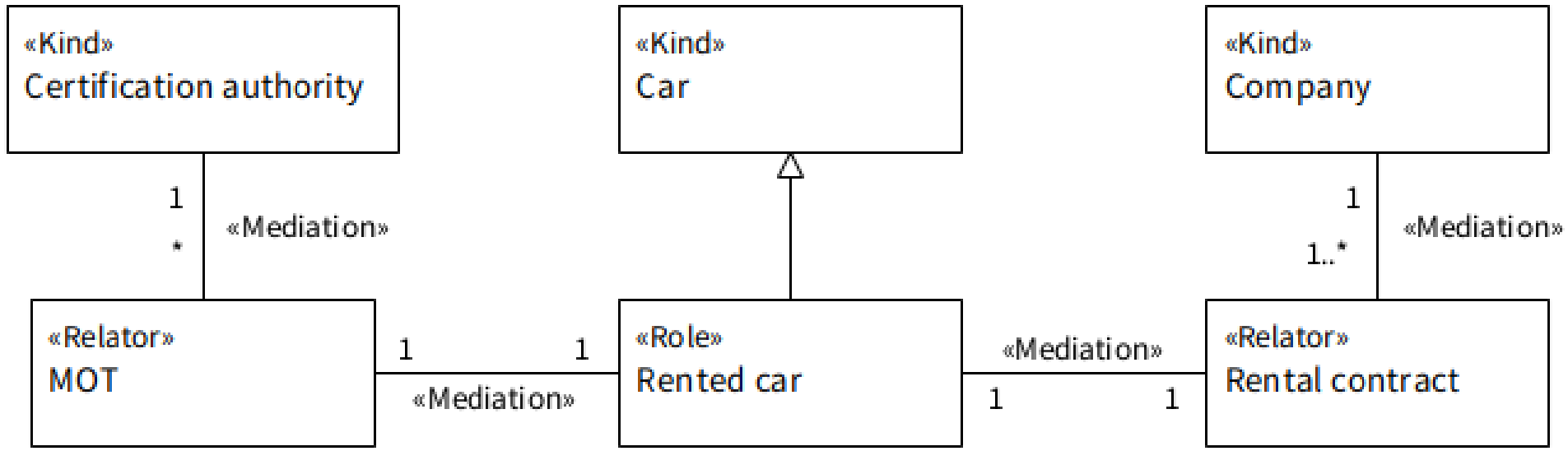


Mixin with the same rigidity (MixRig) - oprava

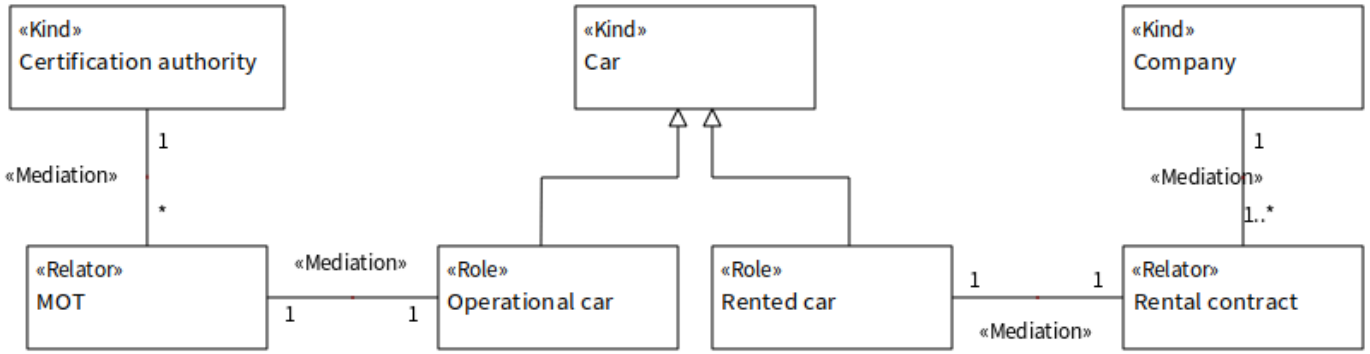


Multiple relational dependency (MultDep)

- **Typ, který je propojen pomocí mediation s dvěma a více různými relatory, které nejsou nadtypem/podtypem jeden druhého.**
- Velmi často důsledek redundance, přílišného zjednodušení, nebo špatné granularity modelu.

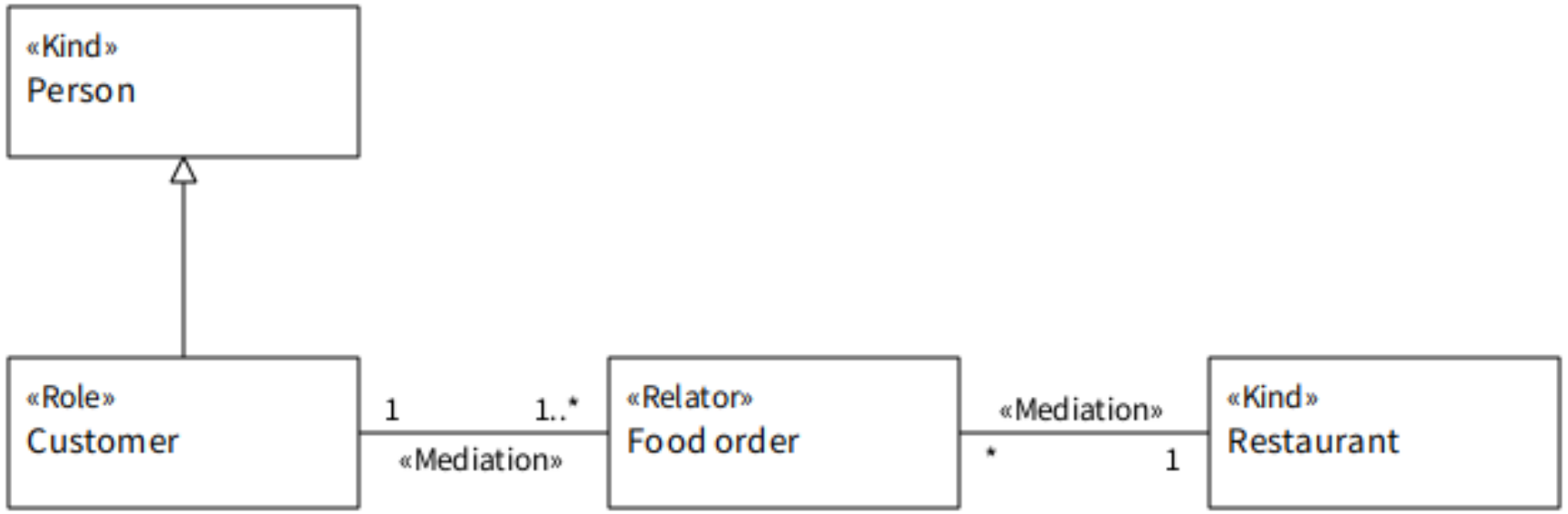


Multiple relational dependency (MultDep) - oprava

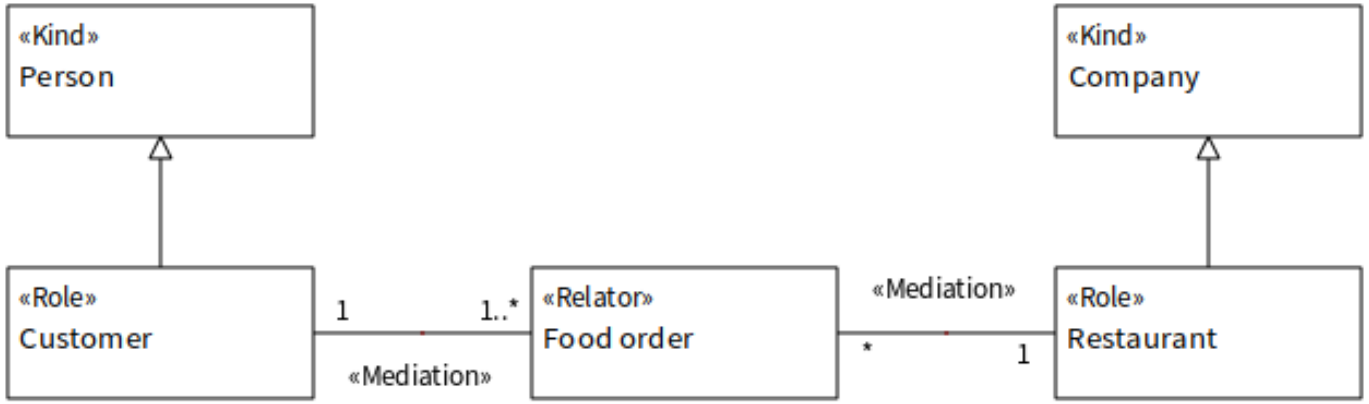


Relator mediating rigid types (RelRig)

- **Relator, přímo propojený mediation asociací s jedním nebo více rigidními typy.**
- Většinou značí chybějící roli, případně rolemixin.
- Může být též znakem toho, že rigidní typ má být stereotypován jako mode.

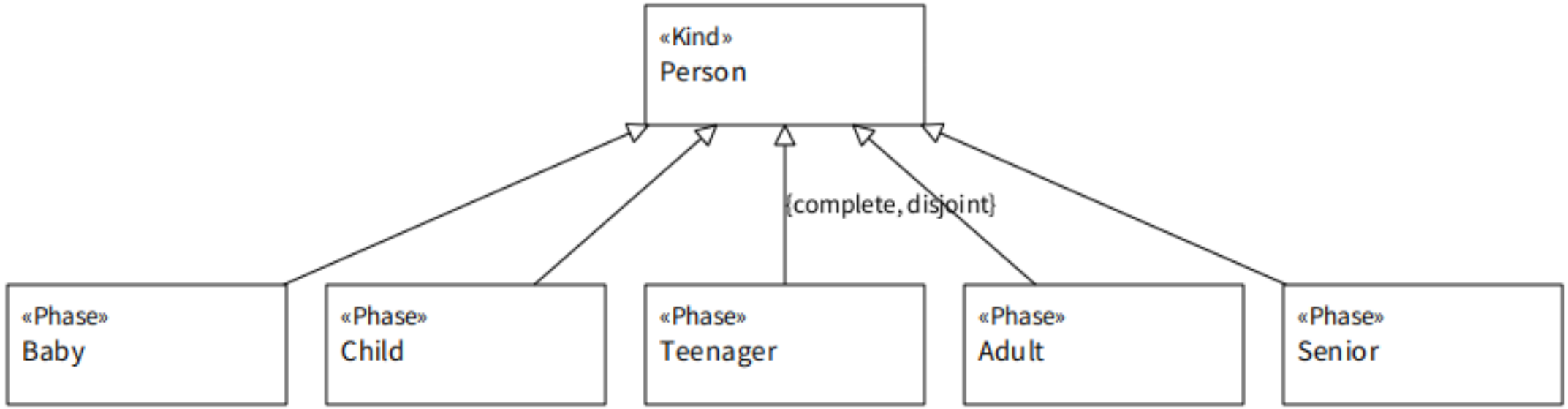


Relator mediating rigid types (RelRig) - oprava

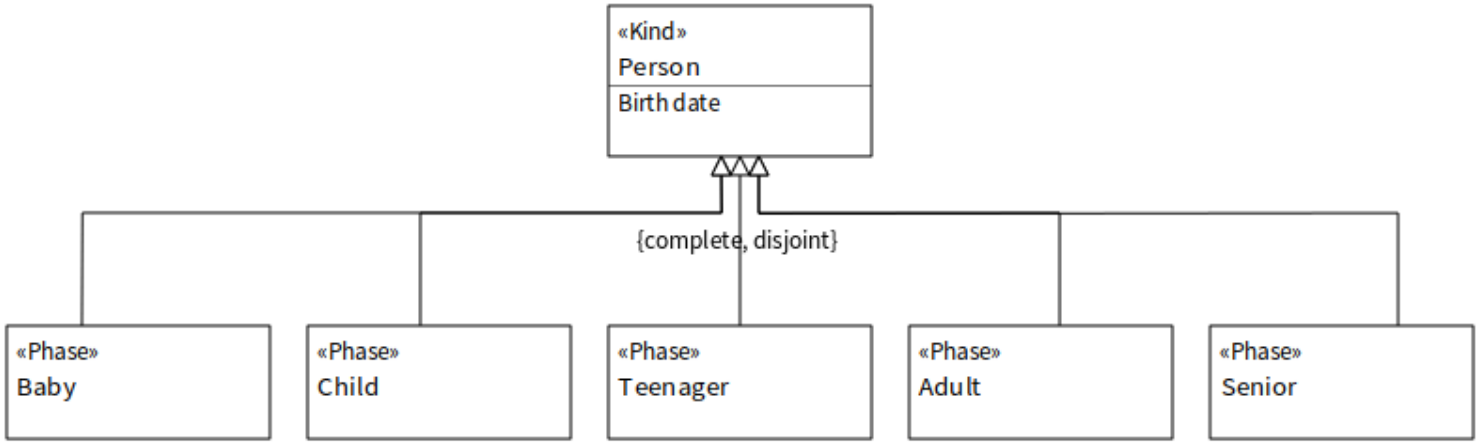


Undefined phase partition (UndefPhase)

- **Množina entit stereotypovaných jako phase, jejichž společný nadtyp nemá žádný atribut, quality ani mode.**
- Phase jsou závislé na změně vnitřního stavu souvisejícího s vlastností nadtypu.
- Tyto vlastnosti lze přiřadit přímo k jednotlivým phase.

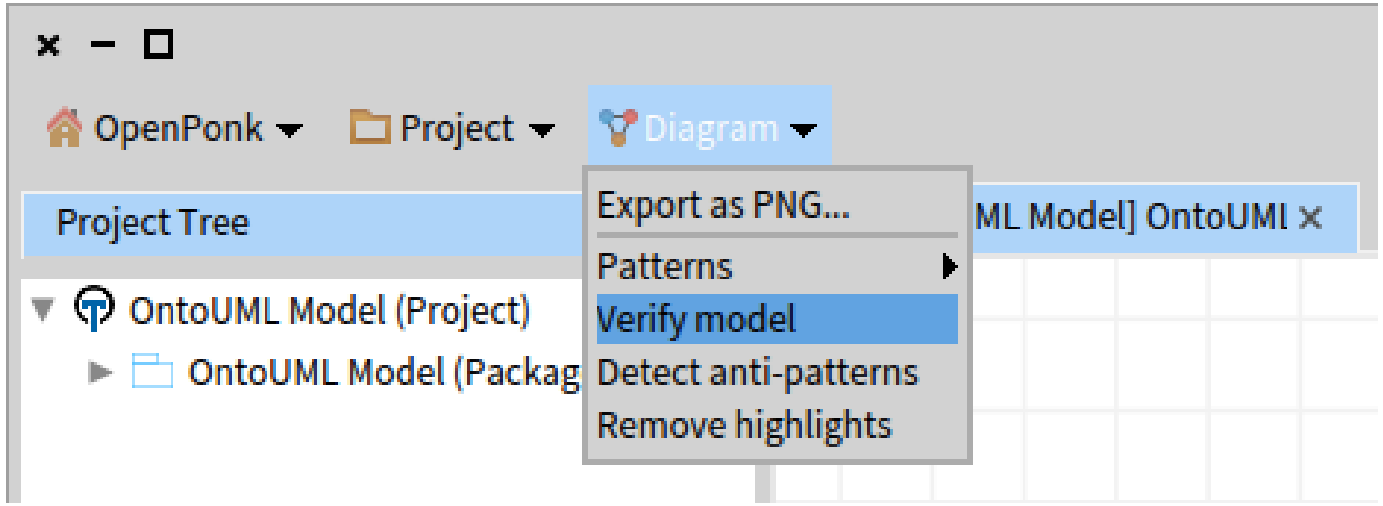


Undefined phase partition (UndefPhase) - oprava

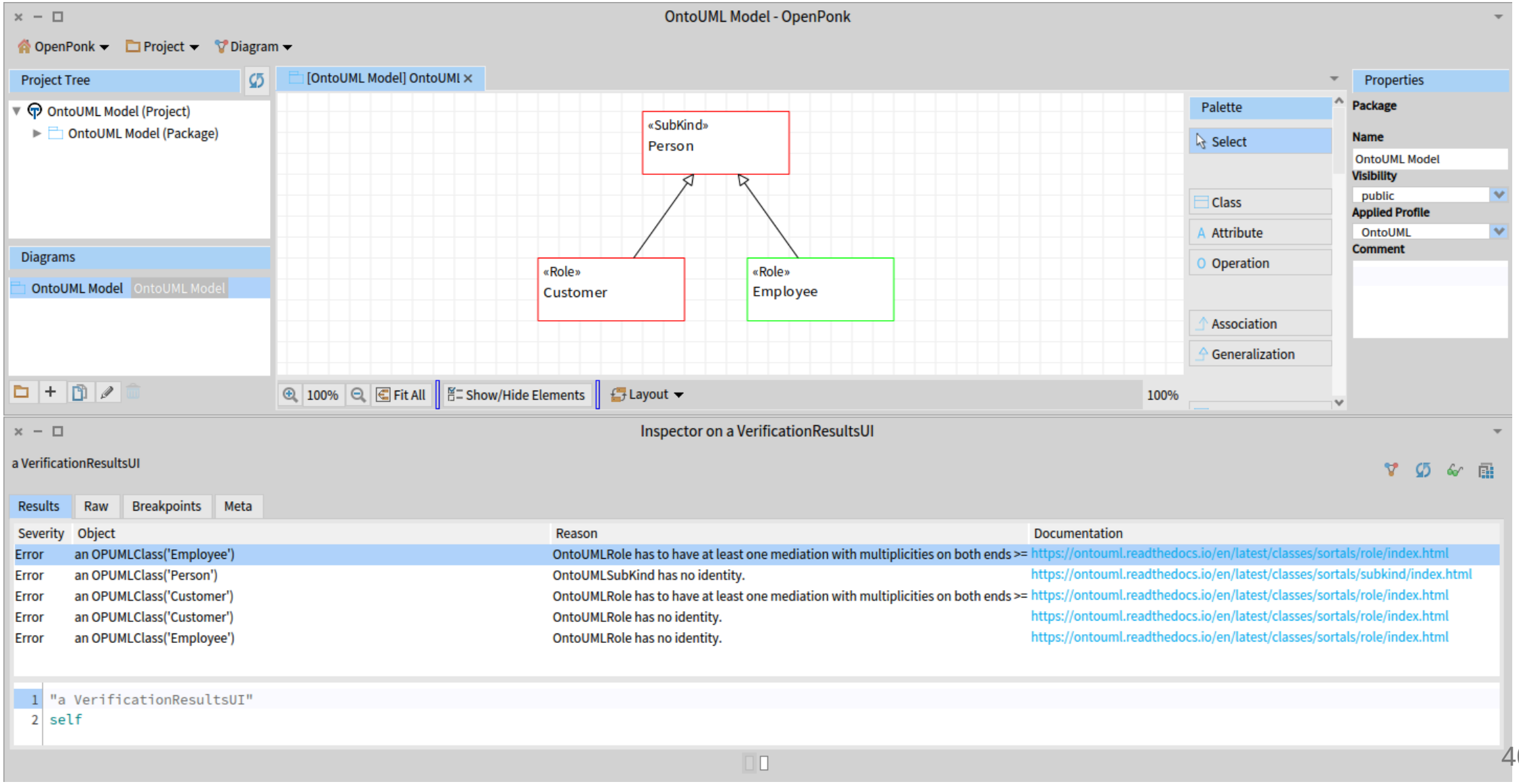


OpenPonk - verifikace a anti-patterny

- Menu "Diagram" v toolbaru hlavního okna
- "Verify model" - syntaktická kontrola, nálezy obarví elementy na červeno
- "Detect anti-patterns" - sémantická kontrola, nálezy obarví elementy do žluta
- "Remove highlights" - vrátí elementům defaultní barvu



OpenPonk - verifikace a anti-patterny



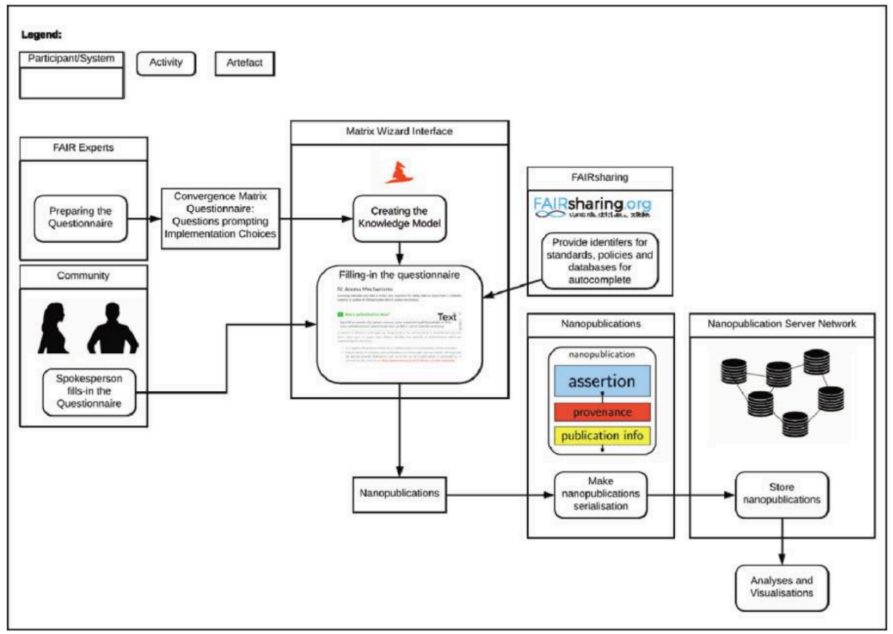
Ukázkové projekty

Projekt 1: Referenční ontologie GO FAIR



Sdílený slovník pro implementační volby FAIR

FAIR Convergence Matrix

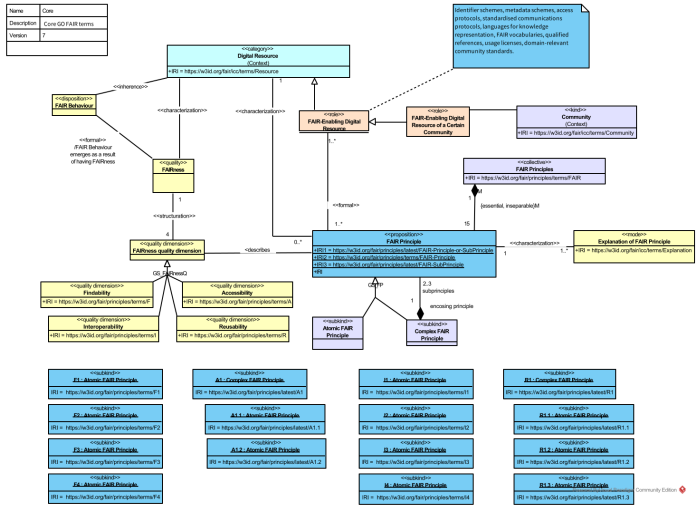


Účel

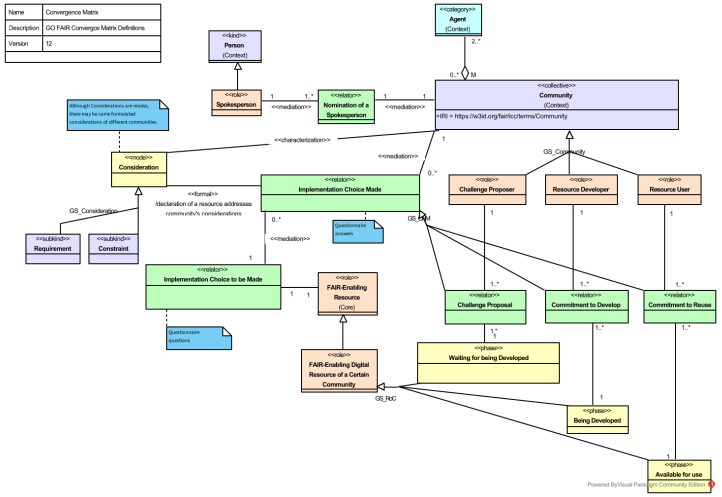
- zaznamenat implementační volby komunity
- opakovaně využívat standardy, politiky a databáze
- publikovat rozhodnutí jako FAIR artefakty

Sustkova, H. P. et al. (2020): *FAIR Convergence Matrix: Optimizing the Reuse of Existing FAIR-Related Resources*. Data Intelligence 2(1-2).

Referenční ontologie je modulární

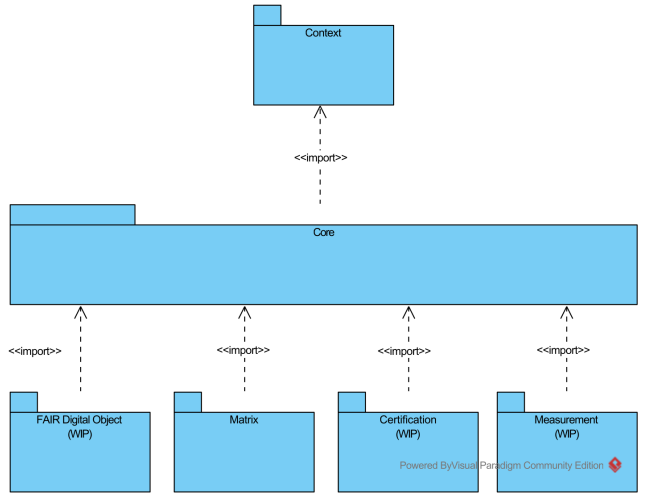


FAIR Digital Object



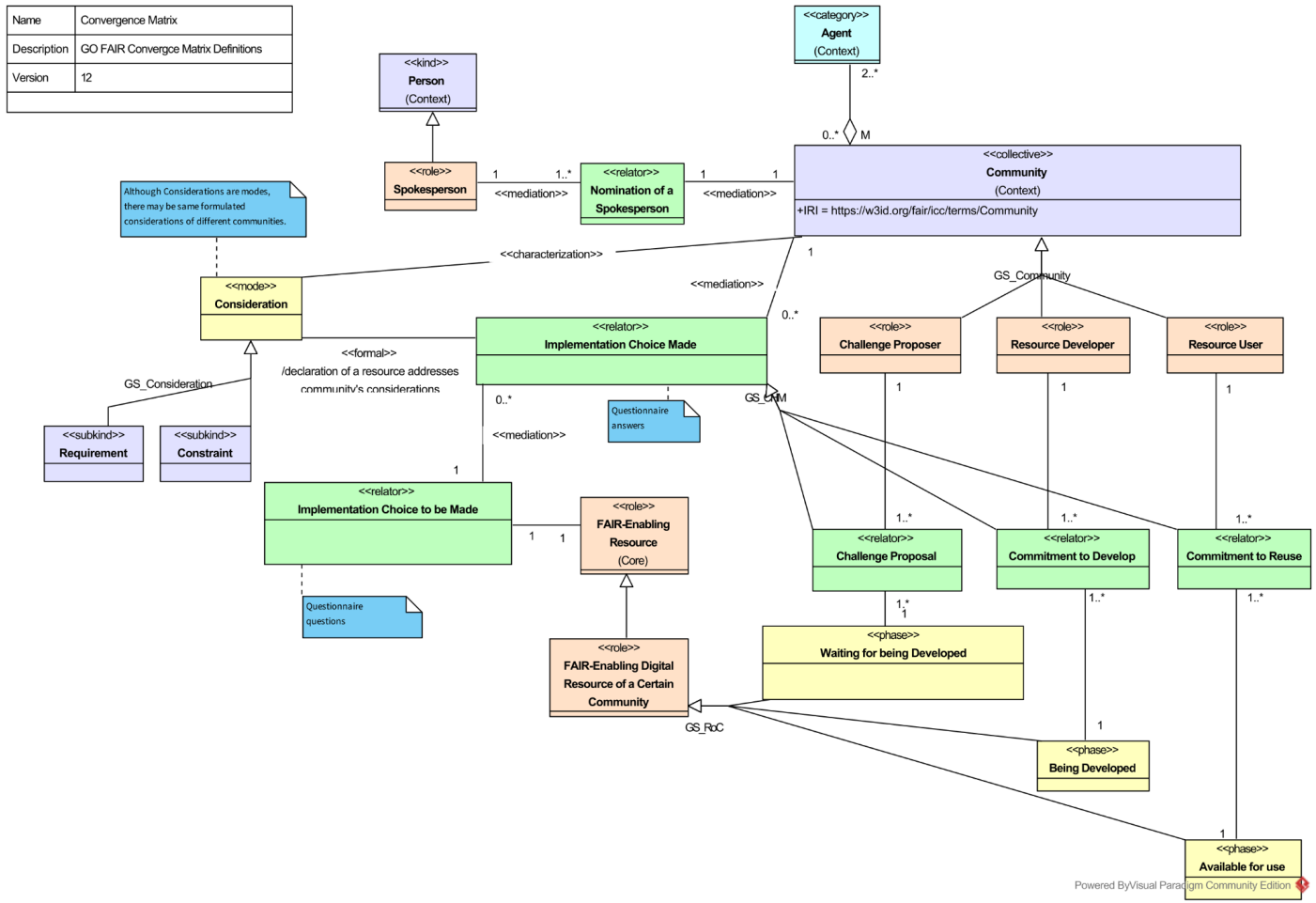
Convergence Matrix

github.com/go-fair-ins/GO-FAIR-Ontology



Sdílené jádro

OntoUML modeluje implementační volby



Model propojuje komunity, účastníky, hlediska, zdroje, závazky a fáze jejich životního cyklu.

Přístup byl později rozvinut do podoby *FAIR Implementation Profile (FIP)*, který využívá FAIR Connect.

Magagna, Barbara, et al. (2024): Ontological Analysis of FAIR Supporting Resources. JOWO 2024.

Projekt 2: Simulátor registru dárců kostní dřeně





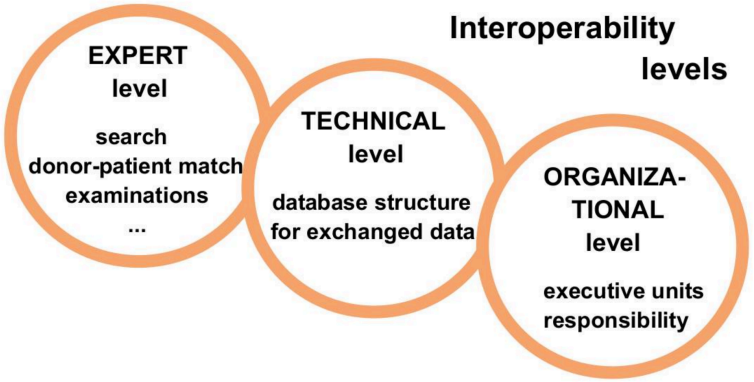


Ontologie zprovozněná prostřednictvím vizualizace a simulace

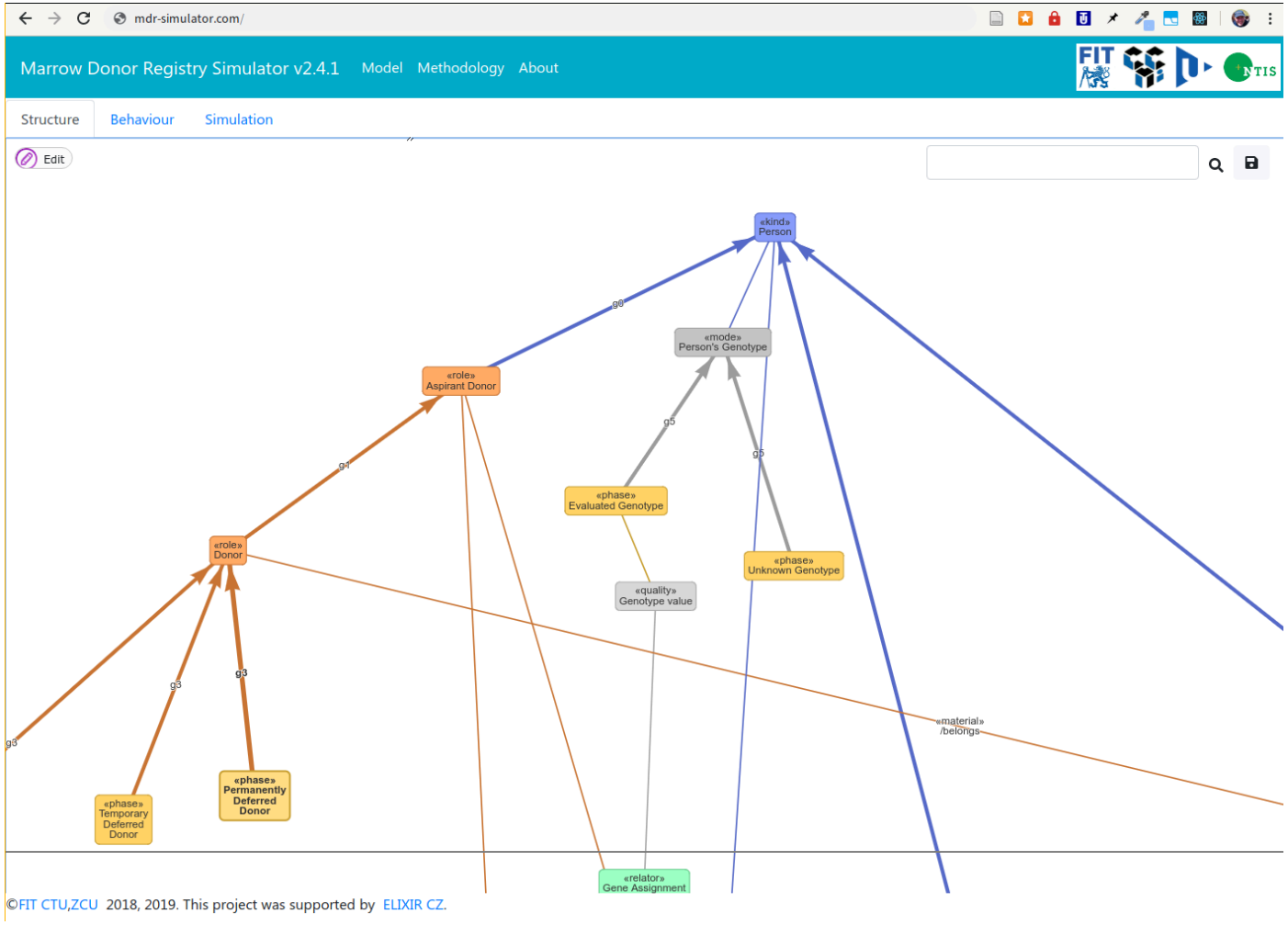
ELIXIR Česká republika | CCMi FIT ČVUT | NTIS

Interoperabilita je problém domény

- Pracoviště registru, například dárcovská centra a laboratoře, musí koordinovat svoji práci.
- Registry také spolupracují na národní i mezinárodní úrovni se specializovanými organizacemi.
- Sdílený význam je nutný na **expertní**, **technické** i **organizační** úrovni.
- OntoUML a UFO definují pojmy, vztahy a pravidla; simulátor je zprovozňuje.

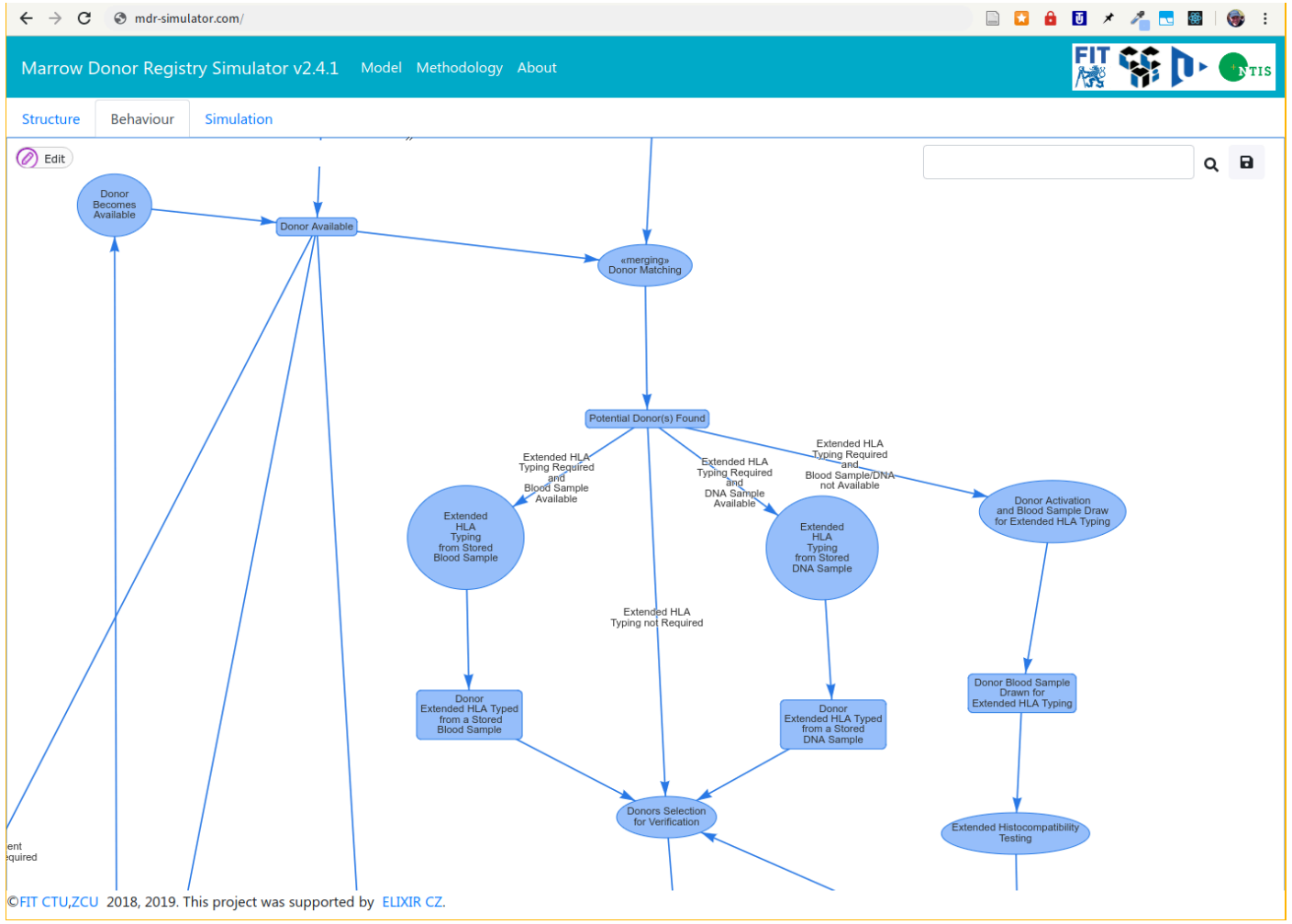


UFO-A umožňuje zkoumat strukturu



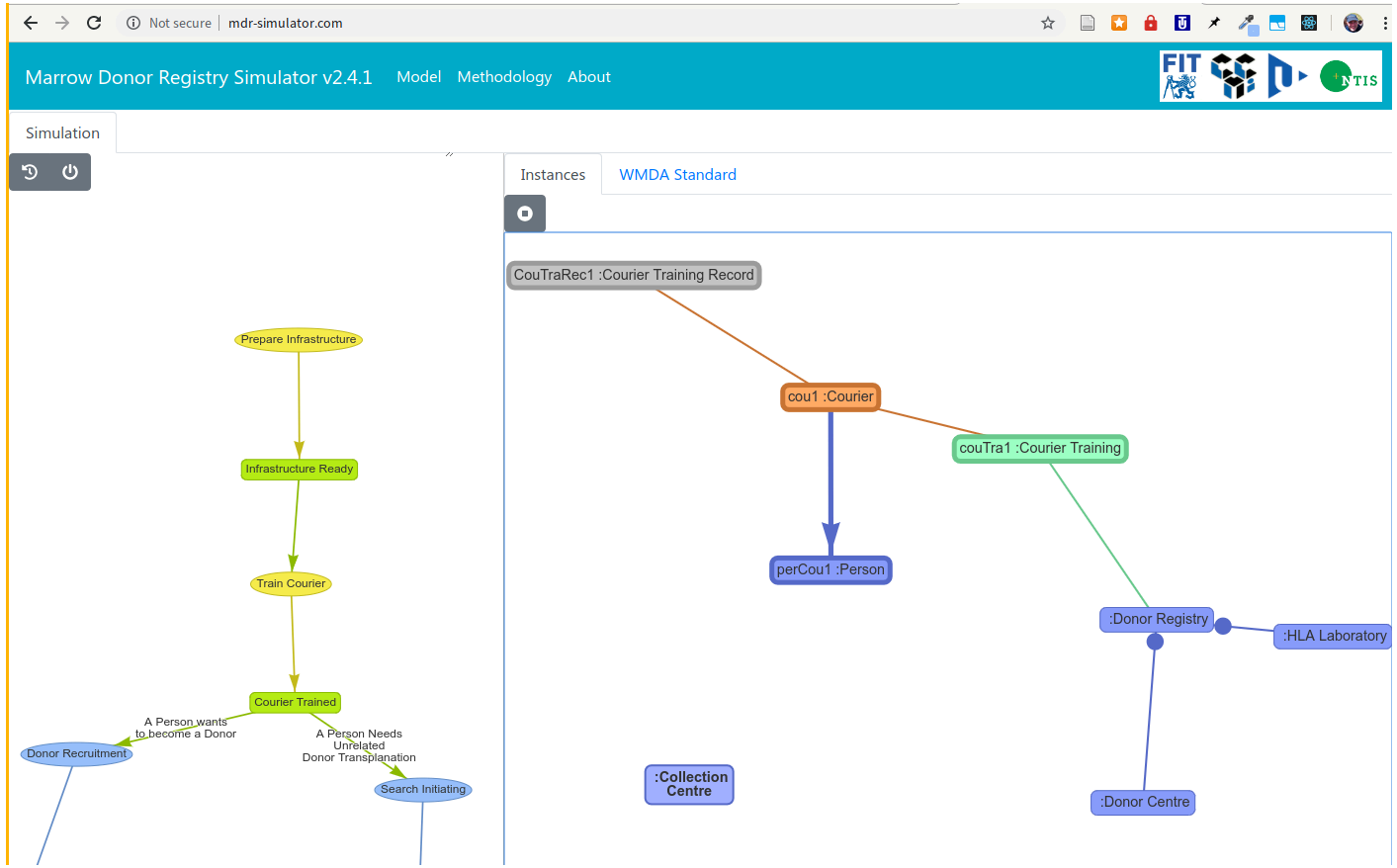
Uživatelé mohou vizualizovat a upravovat role, fáze, vztahy a omezení ve strukturálním modelu.

UFO-B propojuje strukturu s chováním



Procesní pohled propojuje činnosti a události registru s koncepty domény.

Simulace zprovozňuje ontologii



Instance a procesy lze zkoumat společně za účelem validace, učení a výměny znalostí.

Simulátor se používá od roku 2019 a sloužil ke školení stovek zdravotnických expertů v Českém národním registru dárců dřeně.

https://cds-mdrsimulator.zcu.cz

Pozn.: Přestože jsou obsah a chování simulátoru řízeny komplexními pravidly UFO, jeho používání nevyžaduje žádnou znalost OntoUML/UFO.

Projekt 3: Datová platforma pro marketingovou komunikaci

CENTRE FOR CONCEPTUAL MODELLING AND IMPLEMENTATION

FACULTY OF INFORMATION TECHNOLOGY CTU IN PRAGUE

remmark

EVROPSKÁ UNIE
Evropské strukturální a investiční fondy
OP Praha – pól růstu ČR

PRA HA
PRA GUE
PRA GA
PRA G

Ontologické modely jako opakovaně použitelná integrační páteř

Výzva: heterogenní data ve velkém měřítku

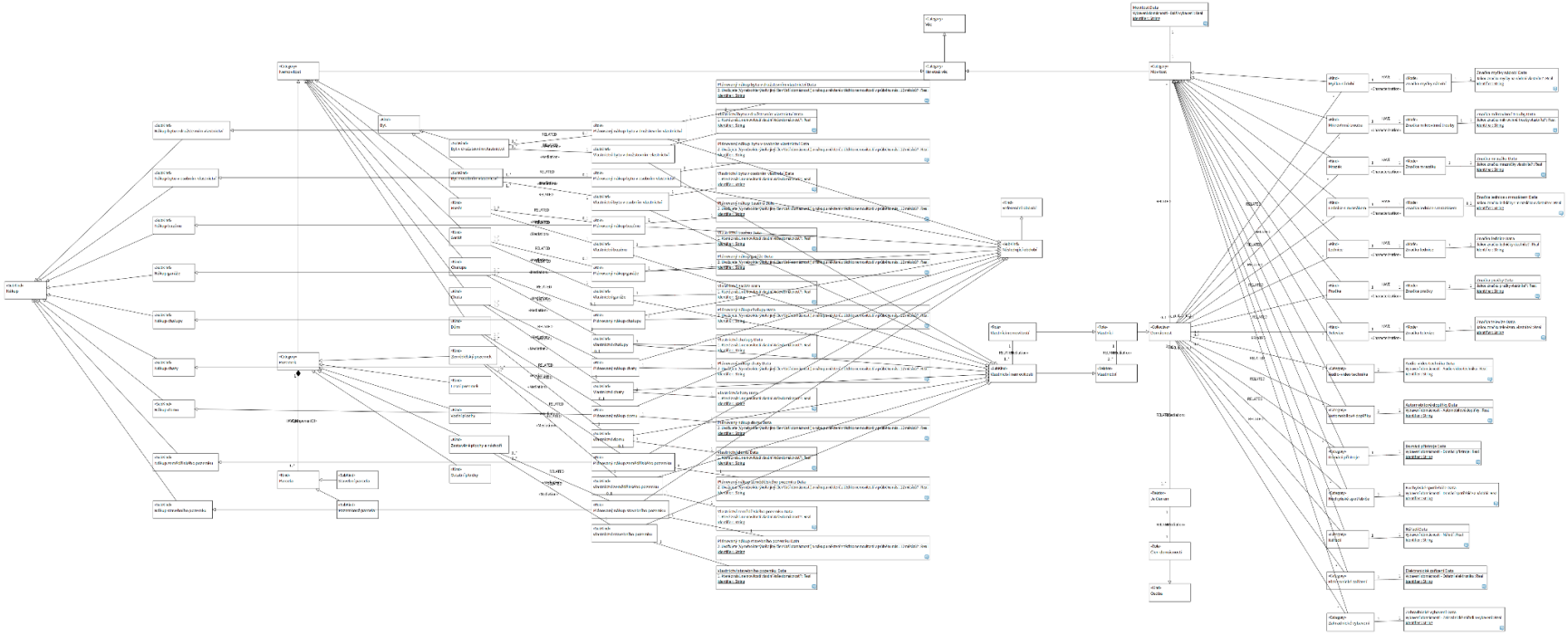
- Integrace uzavřených a otevřených datových zdrojů znamená řešit:
  - různé formáty a kódování
  - různé konceptualizace stejných pojmů
  - synonyma a další sémantické nesoulady
- Konvenční ETL skripty lze opakovaně použít jen částečně, pokud:
  - je třeba přidávat stovky nebo tisíce datových sad
  - se zdrojové datové sady neustále mění
  - data slouží pro analýzy, stahování a aplikační API

Integrační problém je sémantický i syntaktický.

Ontologicky řízená integrace ve čtyřech krocích

1. **Analyzovat domény** pomocí OntoUML a UFO a vytvořit tak sémantickou páteř.
2. **Mapovat datové prvky** na ontologické entity.
3. **Transformovat data** pomocí pravidel odvozených z těchto mapování.
4. **Používat modely** přímo v datové platformě.

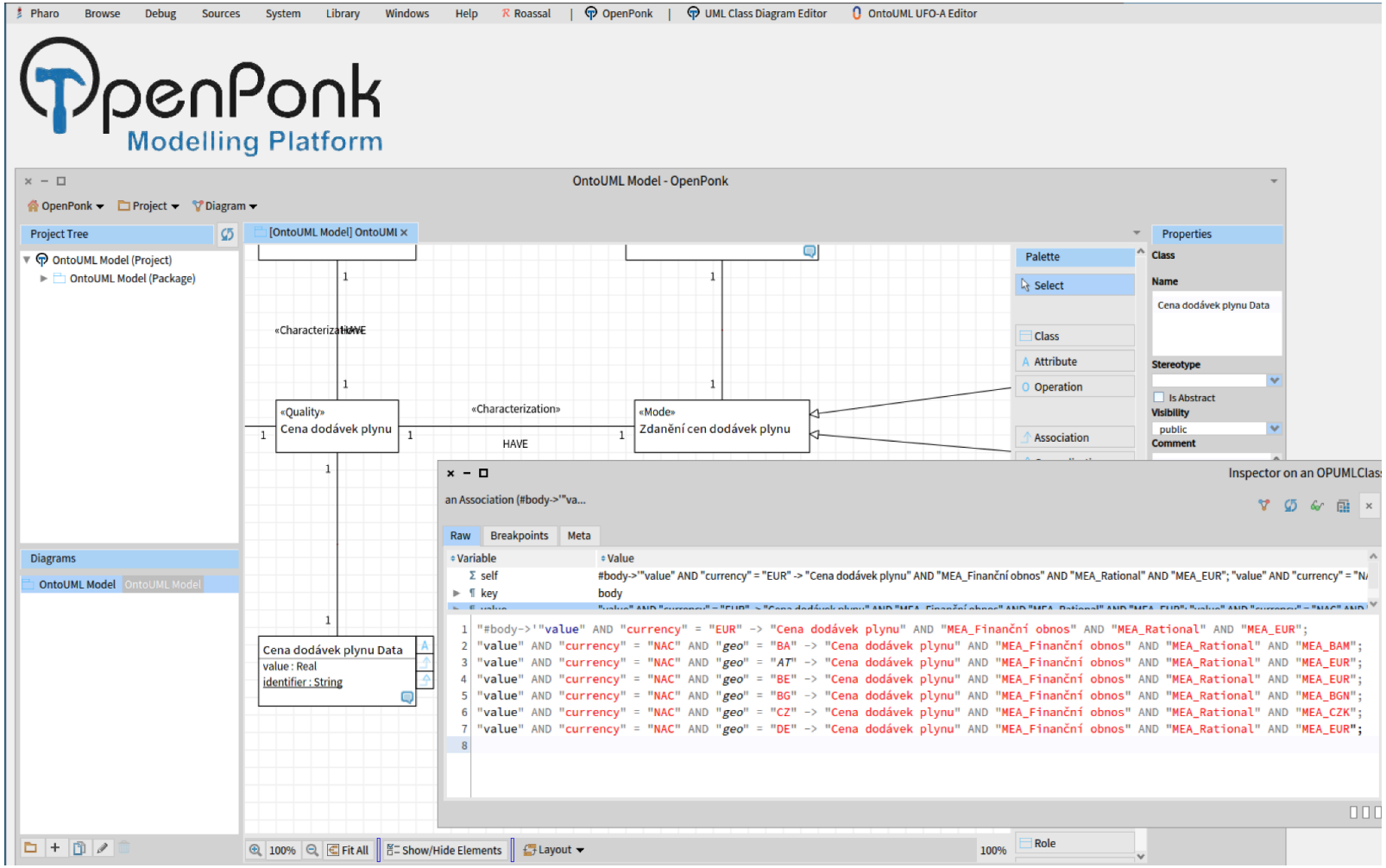
1. Analyzujte každou doménu pomocí OntoUML/UFO



Rozsáhlé modely vyžadují modularizaci a koordinaci týmu.

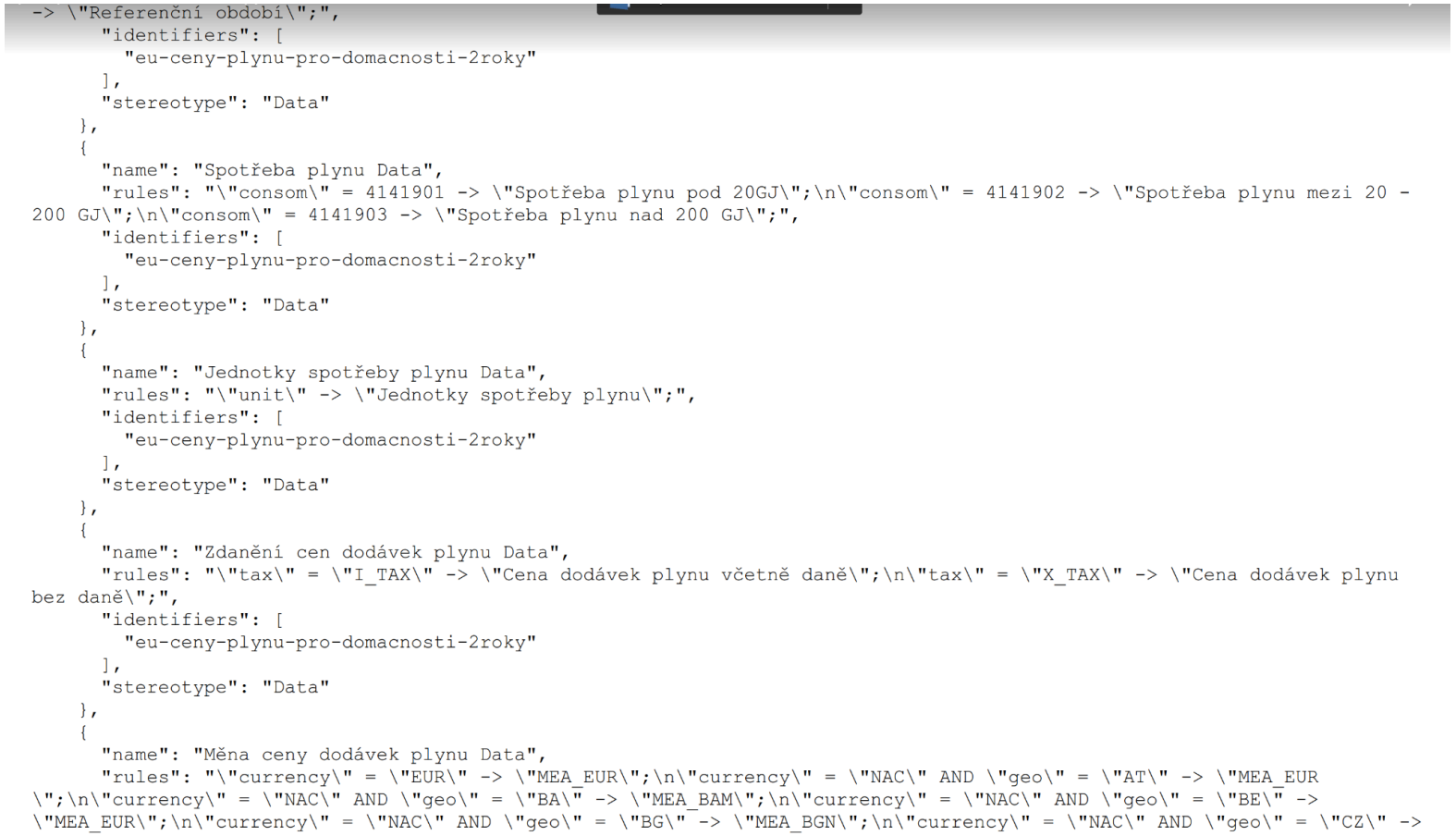
Pozn.: Na modelu pracoval přibližně šestičlenný tým po dobu dvou let; model obsahoval více než 6 800 různých entit a celkem 13 000 entit.

2. Mapujte datové prvky na ontologické entity



Pozn.: Mapování explicitně a strojově zpracovatelně vyjadřují sémantiku zdrojových polí.

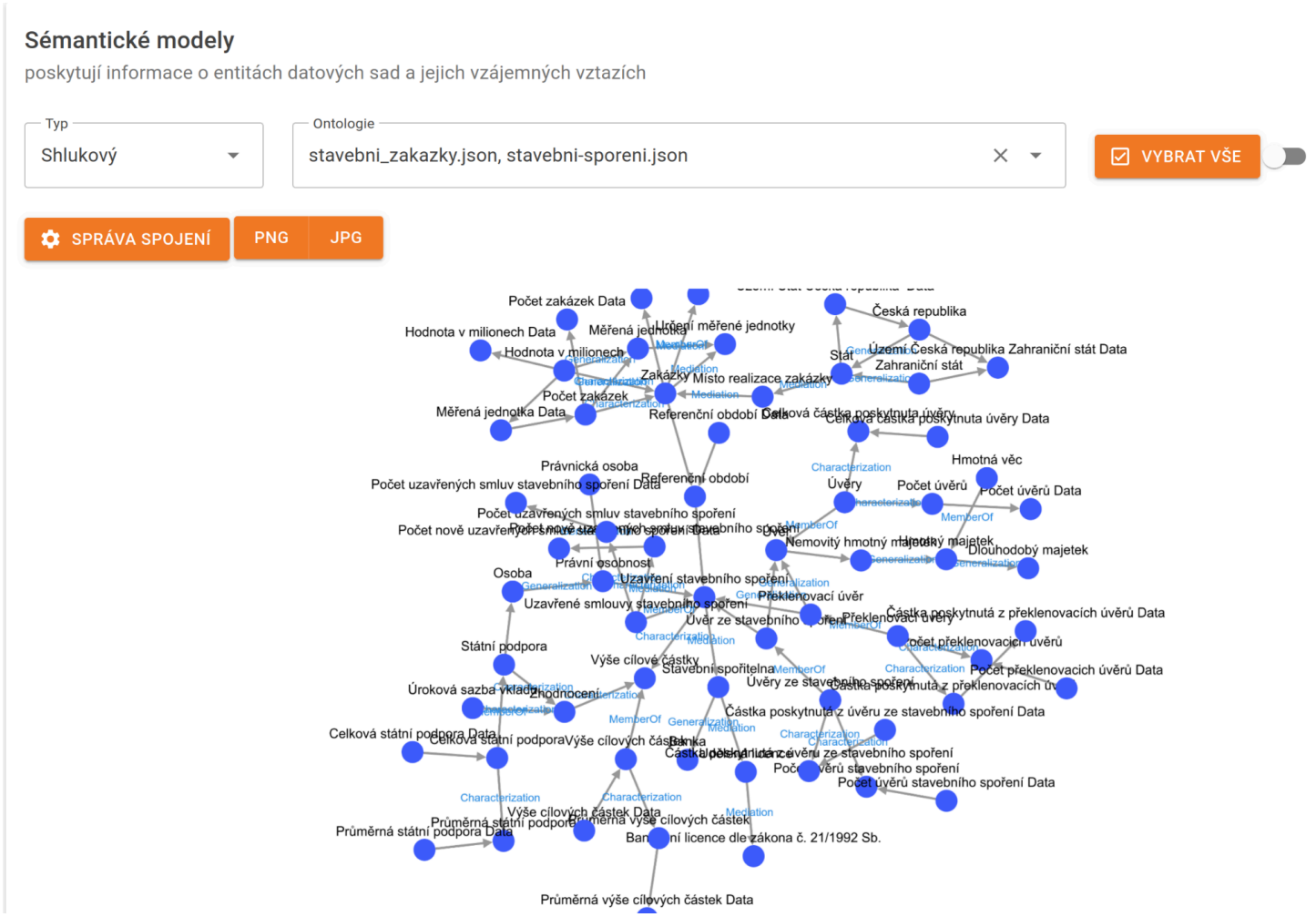
3. Generujte transformace řízené modelem



```
-> \"Referenční období\";",
      "identifiers": [
        "eu-ceny-plynu-pro-domacnosti-2roky"
      ],
      "stereotype": "Data"
    },
    {
      "name": "Spotřeba plynu Data",
      "rules": "\"consom\" = 4141901 -> \"Spotřeba plynu pod 20GJ\";\n\"consom\" = 4141902 -> \"Spotřeba plynu mezi 20 -
200 GJ\";\n\"consom\" = 4141903 -> \"Spotřeba plynu nad 200 GJ\";",
      "identifiers": [
        "eu-ceny-plynu-pro-domacnosti-2roky"
      ],
      "stereotype": "Data"
    },
    {
      "name": "Jednotky spotřeby plynu Data",
      "rules": "\"unit\" -> \"Jednotky spotřeby plynu\";",
      "identifiers": [
        "eu-ceny-plynu-pro-domacnosti-2roky"
      ],
      "stereotype": "Data"
    },
    {
      "name": "Zdanění cen dodávek plynu Data",
      "rules": "\"tax\" = \"I_TAX\" -> \"Cena dodávek plynu včetně daně\";\n\"tax\" = \"X_TAX\" -> \"Cena dodávek plynu
bez daně\";",
      "identifiers": [
        "eu-ceny-plynu-pro-domacnosti-2roky"
      ],
      "stereotype": "Data"
    },
    {
      "name": "Měna ceny dodávek plynu Data",
      "rules": "\"currency\" = \"EUR\" -> \"MEA_EUR\";\n\"currency\" = \"NAC\" AND \"geo\" = \"AT\" -> \"MEA_EUR
\";\n\"currency\" = \"NAC\" AND \"geo\" = \"BA\" -> \"MEA_BAM\";\n\"currency\" = \"NAC\" AND \"geo\" = \"BE\" ->
\"MEA_EUR\";\n\"currency\" = \"NAC\" AND \"geo\" = \"BG\" -> \"MEA_BGN\";\n\"currency\" = \"NAC\" AND \"geo\" = \"CZ\" ->
```

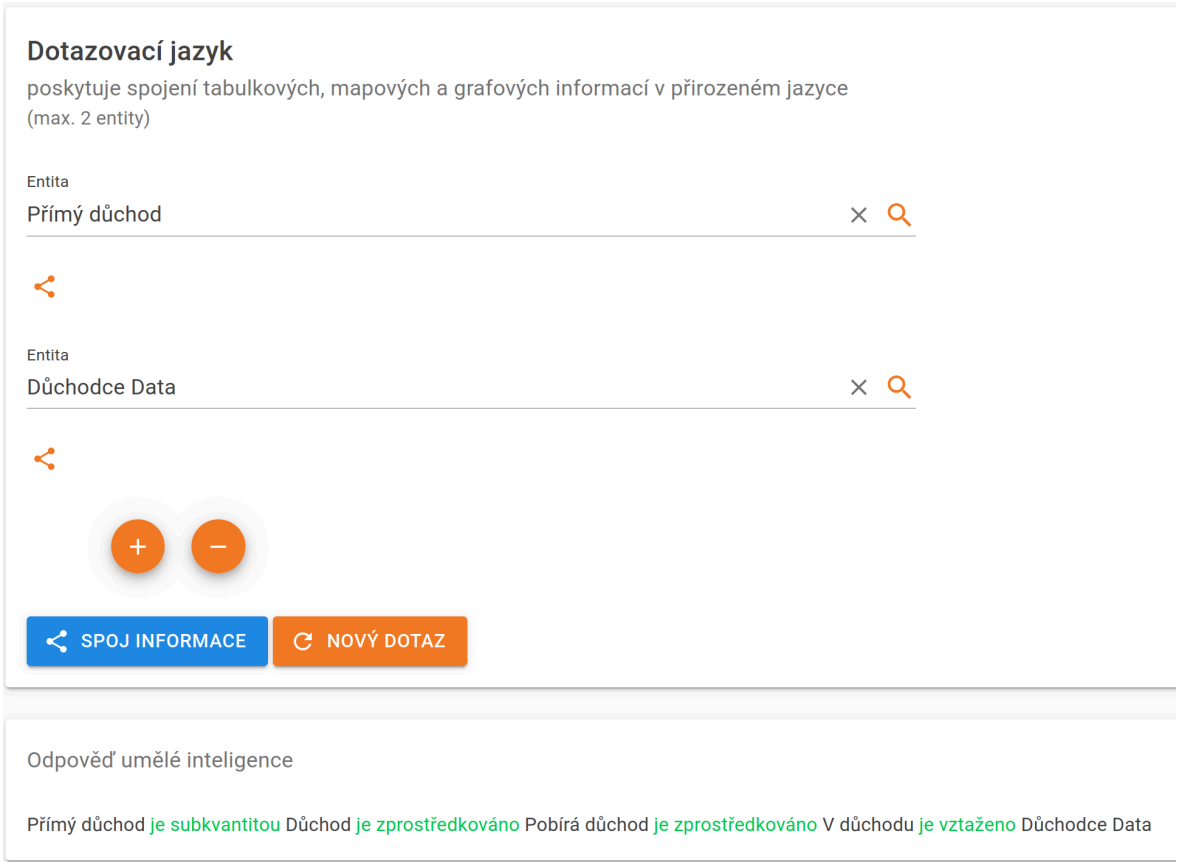
Pozn.: Generovaná pravidla normalizují záhlaví sloupců a hodnoty ve zdrojových datech.

4. Začleňte modely do platformy



Pozn.: Sémantické modely zpřístupňují entity v datových sadách a vztahy mezi nimi.

Dotazování využívá stejnou sémantiku



Pozn.: Rozhraní v přirozeném jazyce může prostřednictvím sdílené ontologie propojovat tabulkové, mapové a grafové informace.

Časté otázky

„Jak poznám, že už mám dost modelování a můžu začít programovat?“

Když model dostatečně vysvětluje pojmy a pravidla potřebná pro zamýšlenou část řešení a prošel ověřením na konkrétních situacích. Nemusíme předem popsat celý svět; můžeme postupovat po částech a model zpřesňovat podle nových poznatků.

Časté otázky

„Nástroj našel anti-pattern. Musím model vždycky opravit?“

Nález je podnět k prověření konkrétního rizika, nikoli automatický rozsudek. Musíme zjistit, jaké nežádoucí situace struktura připouští a zda jsou v naší doméně skutečně nepřípustné. Také model bez nálezů může zachycovat špatně pochopené zadání.

Konec přednášky